MB5370

Techniques in Marine Science 1

Programming fundamentals for Marine Science

Workbook

Version 3.0

Dr Nicholas Murray (Subject Coordinator)

Dr Ben Cresswell (Subject Coordinator)

January 2026

Contents

Introduction

0.1 About the module

Datasets and analyses in marine biology are increasing in size and complexity to address pressing issues in marine environments. Analyses are often a multi-step process that require teamwork and collaboration, and should be reproducible to support repeated monitoring and transparent communication of results.

This module will introduce students to a broad range of concepts that collectively contribute to the important new field of marine data science. Building on an initial basis of developing a core understanding of programming, including data structure and objects, students will develop an understanding of scientific programming and the conceptual organisation of workflows (data and code) to support reproducible analyses of large, complex marine datasets.

0.2 Module goals

At the completion of this module, course participants will:

- Develop the knowledge base necessary to learn and understand new programming languages;
- Gain practical experience in understanding data structures, developing scripts with common programming tools and gain hands-on experience in the software R and Python.
- Understand of the principle of reproducible science;
- Learn how to develop clean, reproducible workflows suitable for developing and running complex analyses of marine data.

0.3 Module preparation

0.3.1 Software

This module requires students to use:

- R and RStudio (the R GUI).
- (Optional) Python via Spyder (Anaconda installation). Ignore the JupyterLab section and simply install Anaconda. We will use Spyder (which comes with Anaconda) as our Python GUI.

If you wish to use your own computer in the module (rather than the lab computers) then you *must* have these programs installed and running prior to arriving at the first computer workshop. See the instructions on LearnJCU for software installation.

0.4 Learning resources

This module has no set text book. However, you may find a range of useful information in the Online Resources section of LearnJCU. These are repeated here.

0.4.1 Overview

There is no set text book for this class. Instead, we will use freely available online materials (including several excellent textbooks) that will be useful throughout the subject. These are outlined below and within the manual for each module.

0.4.2 Books

Some books offer a great overview of programming languages, data structures, variable types and so on. Best bet is to search the JCU library. We will refer to the following free books in this module, which may also be helpful well beyond the end of your degree.

- Grolemund (2014) Hands on Programming with R. O'Reilly Media.
- Wickham, Cetinkaya-Rundel & Grolemund (2023) R for Data Science. O'Reilly Media.
- Healy (2018) Data Visualization: A practical introduction. Princeton University Press.
-

0.4.3 Websites

Stack Overflow. A community resource in forum format where individuals post their problems and others post their solutions. Most useful for very specific problems such as understanding why your code is not running or to learn the background to specific functions. Stack Overflow includes information on main programming languages, including R and Python, as well as software such as Google Earth Engine, QGIS, and ArcGIS

R-bloggers. People write short blogs about different things in R.

RStudio support. The R studio website has a range of resources for learning R. These include cheatsheets, webinars and style guides on how to write neat code. Note RStudio have recently changed their name to posit.

Rseek. Rseek is a meta search engine that searches a broad range of websites focused on R to quickly find and rank information about R.

Software carpentries. Worked examples of a range of different software, including R and Python.

0.4.4 Software help

R help. Each R package includes a help file which follows a standard format. These tend to include specific information about each function, how to run it and nearly always have a standard implementation. Search for R help using the R studio help tab.

R package vignettes. R packages typically come with a so-called vignette. A vignette is written by package developers to demonstrate the core functionalities of their packages. Vignettes are often the most useful and readable of the native R help.

0.5 Module schedules

Refer to the timetable on LearnJCU for class dates, rooms and any other information regarding this module. Any changes to the module will be communicated via LearnJCU.

0.5.1 Lecture schedule

This module is conducted primarily face-to-face in four x 4-hour long computer workshops. However, prior to attending the workshops you must cover the online lecture component (totalling about 1 hour). I will drop a range of hints within the lectures that I will then quiz in person in the workshop, so please make sure to watch them.

Title	**Activities / Content**	**Links**
Lecture 1.1 – Subject Introduction	Subject introduction Module overviews Assessments Timetable Selecting modules	Slide deck Video
Lecture 1.2 – Why program?	Module introduction Purpose of programming Examples of computational marine biology Reproducibility	Slide deck Video
Lecture 1.3 – What is a program?	What is a script Writing code Why so many languages?	Slide deck Video
Lecture 1.4 – Mental models and clear thinking	Code writing principles Style guides Flow diagrams Commenting	Slide deck Video
Lecture 1.5 – R and Python	What makes these different? What should I learn?	Slide deck Video
Lecture 1.6 – Pre-workshop preparation	Installing R	Slide deck Video

before workshop 1	Installing Anaconda Getting the most out of this subject	

0.5.2 Workshops schedule

We have 4 x 4-hour face-to-face computer workshops in this module. The majority of the workshop will be conducted using 'participatory live coding' methods, which means we will work together for the duration of the workshop.

Before coming to the first workshop, please review the module checklist on LearnJCU to ensure you are ready to go when you arrive.

Title	**Activities / Content**	**Links**
Workshop 1 – Foundations	Subject introduction Module assessments Running and quitting Variables Debugging Data types Packages and libraries Coding best practices Building a website	Slide deck Workbook section link
Workshop 2 – Introduction to programming 2	Lists and loops Functions R vs Python practical Markdown languages Assignment	Workbook section link
Workshop 1.3		
Workshop 1.4		

0.5.3 Module preparation and self-directed learning

This module requires some preparation and self-directed learning, consisting of:

- Review all online lectures (*before Workshop 1*)
- Complete module quiz (*before Workshop 1*)
- Finish anything from workshops not finished in class
- Set up and personalise your MB5370 ePortfolio in Google Sites (see Section X below)
- Add material from workshops to your personal website
- Set up and personalise your GitHub profile (see Section X below)
- Add material from workshops to your GitHub pages

Further information about assessment will be provided in workshop 1 as well as in the assessments section of the module folder on LearnJCU. We will cover how to set up your website on Google Sites in this module.

At the completion of this subject you will have a complete website, with an introduction page and then one page for each module that you conducted over the course of the semester. If you wish to, towards the end of the subject we will learn how to register a domain name and have your website indexed by Google.

0.6 Feedback

Early feedback is the only way we can accommodate any requests you have in this module. The best way to do this within the module period is to directly grab one of the instructors during or after the workshops themselves. In addition, early feedback via LearnJCU messenger means we can quickly resolve any concerns. We are always working to improve the student experience in this course, so any feedback you can offer will be warmly received.

Workshop 1 - Foundations

1.1 Workshop overview

Welcome to your first workshop! Whether you are a seasoned field researcher or completely new to computer programming, this module is designed to establish a rock-solid, professional foundation in data science. Today's workshop will ensure you have the fundamental scientific programming skills designed to allow you to gain the most from the remainder of this subject.

In marine science, data arrives in many forms: satellite sea surface temperature arrays, acoustics, coral reef surveys, and shark tracking coordinates. To analyse these data rigorously and reproducibly, we need to move past manual spreadsheet manipulation and embrace programmatic workflows.

This is a 4-hour interactive session. The core material is paced so that absolute beginners can successfully complete it within the workshop time. For students with prior programming experience, each section includes *Deep-Dive Challenges and Extensions* explicitly designed to push your skills and keep you fully engaged for the duration of the session.

This module is aimed at students with limited or no experience of programming. However, for those who do have experience, you will begin to formalise your learning of programming into a structure designed to give you the basis needed to advance your computing skills. In particular, this workshop is designed to bring all students up to a similar level so that the remainder of the modules are achievable.

Workshop schedule

- Hour 1: Navigating the RStudio Ecosystem & Basic Syntax
- Hour 2: Data Types and Variables through a Marine Lens
- Hour 3: The Golden Rules of File Management: R Projects and Directories
- Hour 4: Literate Programming with Quarto & Sourcing Scripts

1.2 Introduction

How can we get computers to work for us in our research as marine scientists? By learning to program, of course!

This module aims to introduce you to *programming* - **not statistics** - so that you can learn useful underlying skills that will help you in your career in marine biology.

We will use R as the basis of learning programming techniques, because it is free, many of you will be familiar with it, and it is the most widely used programming language in the biological sciences. There is also an enormous amount of highly-accessible help for its use in biology and ecology.

It is assumed you have R and RStudio running on your computer or accessible in the computer labs. If that's not the case, please see LearnJCU to follow the installation instructions.

Introduction slide deck.

Pause for thought:
What programming languages have you heard of?
Which programming languages are popular/most used in different disciplines, and why?
What about marine biology?

1.3 The RStudio Ecosystem (The UI or IDE)

Before writing code, we need to understand the dashboard of our vehicle. R is the statistical programming language (the engine), while RStudio is the Integrated Development Environment, or IDE (the dashboard and steering wheel). If you try opening base R by searching for R on your computer and clicking on it, you'll see *Rgui* - the basic IDE that comes packaged with every installation of base R and the native way to interact with it. However, RStudio offers a more advanced workspace that includes built-in data viewers, environment monitoring, multi-pane layouts, and smart auto-completion.

Once installed, you can simply open RStudio like you would any other program on your computer. This is a simple application designed to help you develop R code for running in R.

Note: *You can also run R via the command line or terminal, but we only recommend that if you have a specific purpose (such as scheduling scripts, using a high-performance computing environment or calling R via another program.)*

When you open RStudio, you will see four distinct quadrants or **panes**:

1. **The Source/Editor Pane (Top Left - only visible after opening a file here):** This is your text editor. This is where you write your R scripts and Quarto documents. Code written here won't run until you explicitly execute it, allowing you to draft, edit, and save your work.
2. **The Console Pane (Bottom Left):** The engine room. This is where R actually executes commands. You can type code directly here and hit Enter for instant results, but anything typed here is fleeting and will not be saved.
3. **The Environment/History Pane (Top Right):** Your active workspace. R holds data, variables, and custom functions in memory. This pane gives you a visual inventory of everything currently stored in your active R session.
4. **The Files/Plots/Packages/Help Pane (Bottom Right):** Your utility Swiss Army knife. Use this to navigate your local file paths, view data visualizations you generate, check installed extension libraries, and read documentation via the Help tab.

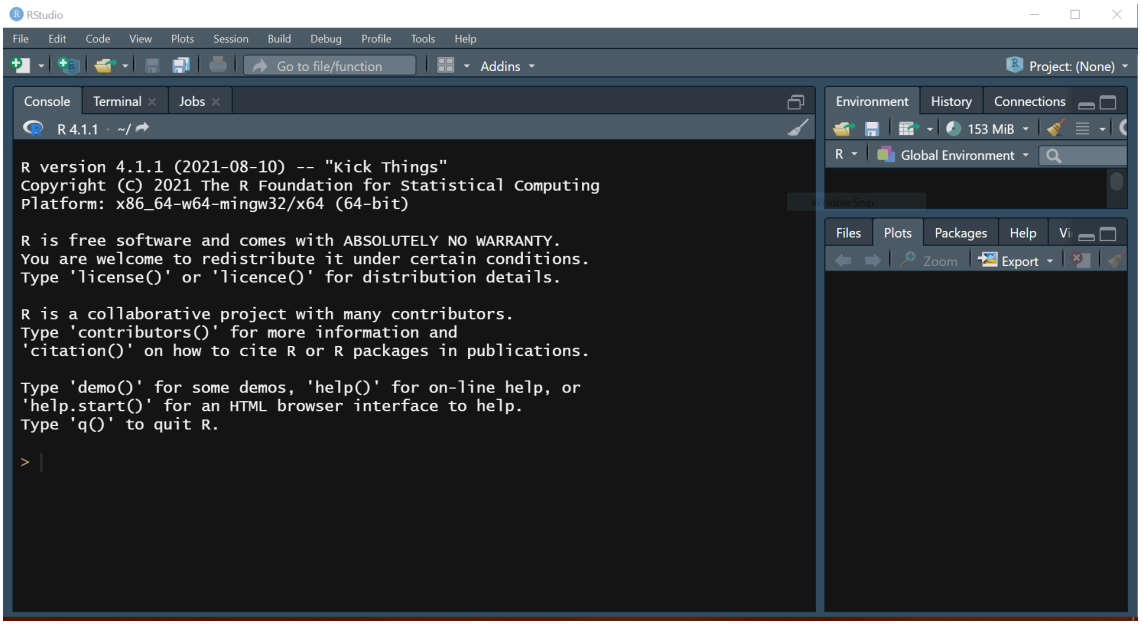


Figure 1. RStudio opening panes.

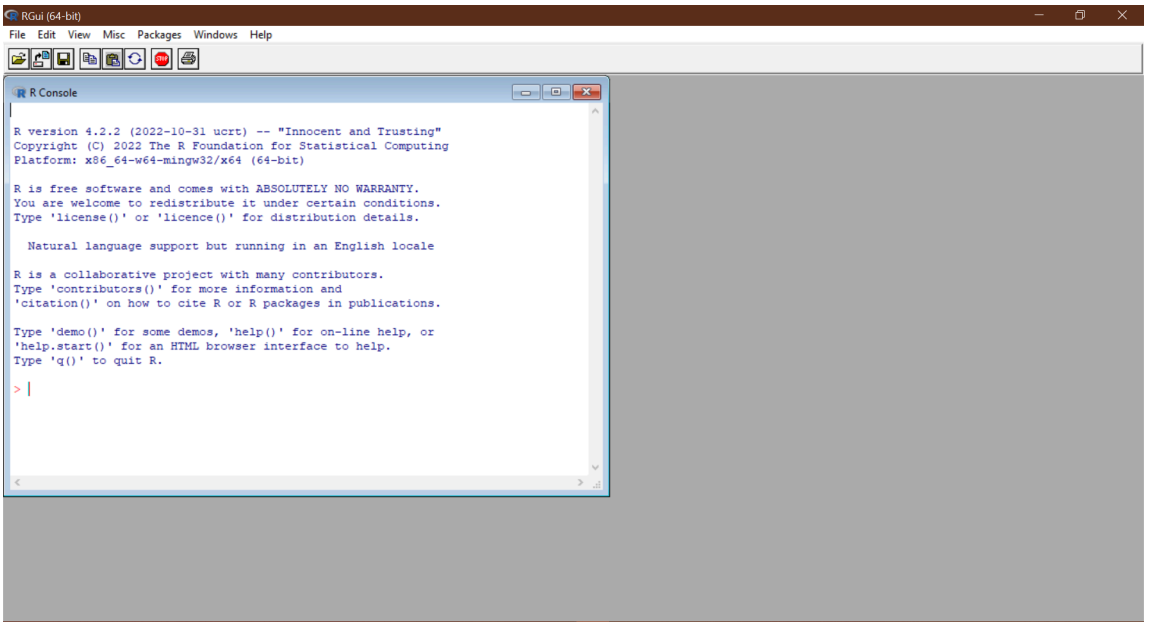


Figure 2. RGui – the original GUI (Graphical User Interface) that many used before RStudio arrived.

Note that RStudio doesn't come with R, it simply helps you interact with R. **You'll need to install both R and RStudio on your computer.**

The other panes are additions that RStudio includes to make your life easier and we'll learn about these later in the module.

1.4 R Syntax and working in scripts

In this section we'll cover the basics of setting up scripts, writing good code and practice sending commands from the Source editor down to the Console.

Above the Console is the 'Source' pane where you write the text code that R can then run. The source pane allows you to access files where you can develop and save your code. The most basic of these are Scripts. People work in scripts because they describe a set of coded instructions to run in sequence all at once, they can be saved and improved upon at a later date, and run again and again.

To launch a new script, go to the menu File > New File > R script to open an empty script in the source pane. Note all of the different file types you can create under the New file option (Rmarkdown, Quarto etc - we'll come to these later!)

Also note that when you create a new script the name of it will appear as something like "Untitled1". This is similar to working in Word or Excel, when you create a new file it isn't yet saved or even named. We could just close/delete this script right now, and RStudio won't warn us of anything, because we haven't made any changes (yet!). Let's go ahead and start adding some simple code to our script.

The Console Calculator

At its simplest, R can act as a calculator. Type the following command in your script at the top and press Ctrl+enter (or Cmd+enter on a Mac, or click the "Run" button at the top) and you'll get a result. Notice how both the command and the result actually appear in the Console pane:

```
# Basic arithmetic (e.g. wrangling sea surface temperatures)
24.5 + 1.2
32.0 / 4
```

This is because, although the script contains the code, it is actually processed or "run" in the Console. See here for a schematic of how this works. Moving forward we are obviously going to increase the complexity of computation that we'll ask R to do, but the mechanics stay the same!

Commenting and style guidance

Notice how in the above code, there is a line that didn't run? The one beginning with `#`.To avoid R interpreting lines of code that we don't want it to, we use a practice called "commenting". R treats the character `#` in a special way - it won't run any text that follows it on a line of code. Other programming languages may be the same (`#` works in Python too) or different - JavaScript (e.g. for Google Earth Engine) comments are via "`//`" .

Comments are super useful for making notes to yourself, setting headings for sections of code, recording little annotations and to 'block out' anything you don't want R to interpret (such as old code that you want to keep for reference, but don't want to run).

In the lectures we introduced mental models, clear thinking and, importantly, how style guides can help you reduce error, improve your ability to understand your code and share it with others.

This is a style guide for R, written by the tidyverse crew, and is one suggestion of how to style your work for clarity, readability and usability. By maintaining a good style, you'll find your scripts are easier to read and debug. More importantly, in a few years time when you revisit them, you'll be quicker to understand what your own code actually does. Let's start implementing good style by giving our script a header section.

Let's incorporated some useful information to the top of our script.

```
#-----------------------------------#
 # MB5370: Techniques in Marine Science 1
 # Programming Fundamentals
 # < YOUR NAME HERE >
 # <  DATE >

 #-----------------------------------#
 # Workshop 01. Introduction ####
```

It is generally recommended to use informative and long-format comments rather than short cryptic ones. This saves you wondering what you were talking about next time you open your script.

We'll use comments to break up our script and follow the structure of this manual, such as this:

```
# R syntax and working in scripts ####
# This section introduces us to R by running simple calculations
inside a script.
```

We can use comments as headers to sections (e.g. `# header text ####`) , commented lines to break up long scripts into chunks (e.g. `#------`), make little notes on a line of code (e.g. `# a useful comment`) or to record results of code (`## result`). Get yourself familiar with style guides and follow a practice that works for you.

Indexed outputs

Run the below calculation. The results which will appear in the Console:

```
> 2 + 1
 [1] 3
```

The `[1]` next to your result is simply an *index* to the first value of your result. This number, which identifies individually a part of your result, can become important when you get lots of results from your code.

Use the `:` to make a sequence of numbers (every number between two numbers), which returns 30 results.

```
> 1:30
 [1]  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21
[22] 22 23 24 25 26 27 28 29 30
```

Note that each value of your result is assigned an *index*, which is displayed only on each new line of output. Have a go at resizing your console pane and see the effect it has on the index values that are displayed:

```
> 1:30
[1]  1  2  3  4  5  6  7  8
[9]  9 10 11 12 13 14 15 16
[17] 17 18 19 20 21 22 23 24
[25] 25 26 27 28 29 30
```

For now this feature is not that useful, but it can be important if you want to pull out a particular result. For now we'll just ignore those *index* numbers in brackets.

Incomplete lines of code

If you write a command in the console that's incomplete, R will wait for you to complete it. This is one of the most common early syntax errors that novice programmers make.

```
> 6 *
 +
```

Here, the `+` is reminding you to finish writing your command. Just type in the next bit and click run (or enter) and you'll get your result.

*Note the * is multiplication in the programming world, rather than "x".*

```
> 6 *
 + 2
 [1] 12
```

A great thing about R is that if you have a syntax error and try to run your code, R will be very obvious that your command doesn't work.

```
> 6 % 2
 Error: unexpected input in "6 % 2"
```

Working in a script means you can just go back and fix it! Neat! Do that now so that your script will still run.

Functions and arguments

An exceptionally powerful aspect of all programming languages is that they include many built-in *functions.* These allow you to do many different things, ranging from simple things like rounding numbers to sampling numbers or reporting simple things about an object, like its type.

Here is a full list of R's built-in functions.

A simple example is rounding a number. We can use the built-in functions of `round` and `floor`.

Don't forget to give this section of your code a new title section - keep your code readable!

```
years_old <- 25.7
round(years-old) # rounds up
floor(years_old) # rounds down
```

Each function has *arguments*. For instance, round has an argument that lets you specify how many decimal places you want to round a number to.

```
years_old <- 25.765
 round (years_olld, 2) # comma after the object to
specify argument
 [1] 25.76
```

To discover the options for each function, you can query function arguments using the `args(xxx)` function or use a question mark for shortcut to the R help pane.

```
?round # go to help
```

```
> args(round) # use args in the Console
 function (x, digits = 0)
```

You can also use the "Help" pane in the lower right hand corner of your R Studio terminal to search function help pages.

Objects and the assignment operator

Computers do not understand you in the way another human would. Calling something a “number” doesn’t mean it will make it a number type. This section will introduce us to the types of data that programming languages know.

Before we move on, let’s name the next section in our script using comments:

```
# Objects and Assignment
## This section focuses on understanding how data is stored in R and why
that matters.
```

How can I store my data? This is where programming your workflow in a script really begins to show its value. Programming languages allow you to store things as *objects* (sometimes also called a variable).

Storing data inside an object gives you the powerful option of calling it back later on to do more things with it or to it.

To save data for later use, we use the **assignment operator**, which looks like an arrow pointing to the left: `<-` . This takes whatever is on the right-hand side and saves it inside an object named on the left-hand side.

```
# Saving a single value
coral_count <- 42

# Saving a vector of multiple fish lengths (in mm)
fish_lengths <- c(124, 152, 98, 221, 146)
```

You can now see these stored in the ‘Environment’ tab in the top right, which displays everything R has in its current session.

When describing code using the assignment operator, we typically say “this gets that”. So if we were to say the above aloud it would be “coral count gets 42” or “fish length gets…” etc.

What about `=` ??

Technically, you ***can*** use = for assignment in most cases, but the R community universally prefers `<-` to cleanly differentiate setting variable values from argument declarations inside functions. Use the shortcut Alt + - (Windows) or Option + - (Mac) to quickly insert `<-`.

Manipulating objects

Whenever R encounters an object, it will substitute it with the data saved inside the object, like this:

```
> coral_count + 1
 [1] 43
```

```
> coral_count + coral_count
 [1] 84
```

And also note that Cases Matter!

```
> Coral_Count <- 1
> coral_count + Coral_Count
 [1] 43
```

Object naming rules

There are some important rules to naming objects in all programming languages. In R, you can't use a number at the beginning of an object name and you can't use some special symbols, including spaces. Object names are also case sensitive and some things are reserved only for the program to use (like `for` and `if`). See the reserved words here.

Let's explore some examples:

Try naming an object with a number at the beginning…

```
> 01_age <- 25 # starts with a number
Error: unexpected input in "01_"
```

```
> !_age <- 25 # no special symbols
Error: unexpected input in "!_"
```

```
> coral count <- 25 # no spaces
Error: unexpected symbol in "coral count"
```

We can technically circumvent this limitation, by using `back ticks` (top left corner of ANSI keyboard, bottom left corner of ISO keyboard):

```
> `coral count` <- 25 # spaces used but with back ticks
```

Using backticks is useful in some situations, however, it is better to use informative object names to make your code readable (as recommended by style guides). If you need to add detail to a title then use underscores or hyphens when possible. Something like `age_in_years` or `age_yrs` is probably better than just `age`. It's also good to be consistent. For example, if you use underscores then stick with them rather than changing to a hyphen.

Debugging code

Professional coders such as software engineers *spend 40-60% of their time debugging*, novices spend much more. Debugging is the process of finding and fixing errors in your code, and is perhaps the most important skill you can learn to reduce the effort it takes you to program.

In these workshops, code blocks will sometimes contain deliberate errors. This is designed to break your reliance on blind copy-pasting and force you to read, interpret, and debug error messages. Error messages often give you the hint you need to understand what's going on.

Sometimes it can be worth copying the error report and pasting it into Google, which is a quick way to find help on sites such as StackOverflow. Note that everyone does this simple thing, pasting an error into Google to work out what is going on, so learn to read and understand your error messages!

Exercise: Copy the following block into your script and run it. Inspect the error message that appears in your console:

```
# Field survey data
quadrat_area_m2 <- 0.25
number_of_quadrats <- 16
total_area_surveyed <- quadrat_area_m2 * number_of_quadrats

# Let's print out the result
print(total_area_surveyd)
```

Task: Read the console output. What does the error message tell you? Fix the code in your script so it successfully prints out the total area surveyed.

1.5 Packages

How can you use software that other people have written, perhaps for some special purpose like using a particular statistical model or plotting a figure? The answer is by finding and loading their packages (or libraries).These are called libraries in Python and packages in JavaScript and R, sometimes also called modules (we'll refer to them as **packages** in our subject from now on).

Packages are a collection of functions that can be downloaded and used by you. They are generally made by researchers, professors, statisticians and programmers, who then make them free for you to use.

Currently, there are nearly 20,000 packages available for R! Find them all here. And here is a great list of curated packages that are very useful and widely used, so you don't have to filter through all 20,000 packages to find what you need. Chances are this curated list has what you need.

To make your own code an R package, you need to write a set of functions and package them up into a way that they are reliable to use on any computer. They need to include specific information and must be formatted in a particular way. They are then reviewed and published on CRAN (The Comprehensive R Archive Network)

Installing and loading packages

To use R packages, you must first download or **install** them. Once installed on your machine, you then need to **load** them, for each new R session. Although there are thousands of packages available on CRAN, there are some packages that are widely and commonly used in almost any data manipulation task in marine science. One entire family of packages is the tidyverse. This collection contains packages specifically designed for wrangling data, making awesome figures, manipulating dates/times and much much more!

Exercise: installing and loading tidyverse

To install the tidyverse on your machine, copy and paste the below code into your **Console**. Note: code to install packages should never be hardcoded into script, as this runs the danger of repeatedly installing the package over and over again, each time the script is run. At best, this is inefficient, at worst you can accidentally overwrite a version of package, which can be damaging in certain situations.

The function to install packages is unsurprisingly `install.packages()`!

```
install.packages("tidyverse") # download and install
```

Once you've installed your package you won't need to again on the same computer. However, if you switch to a new machine you may need to do a fresh install, so be aware of that.

So now you have tidyverse installed on your computer. Next we need to load it in order to be able to access and use the packages contained within.

```
# library(tidyverse) # load into current session
```

Alternatively, if you knew you only needed one package from tidyverse, for example dplyr or ggplot2, you could load these individually:

```
library (dplyr)
library (ggplot2)
```

Getting help with packages.

Every R package must include a help page for it to be accepted to CRAN. The easiest way to access that help is simply use a question mark like we did for the base R function `?round.`

Every help page follows a similar format, which includes a description of the function, how you use it (showing all arguments you can pass to the function) and some examples.

Each R package also includes a vignette, which is a document that gives examples about how you would use the R package.

RStudio gives a nice easy access to all of this help documentation in the help tab in the bottom right hand pane.

1.6 Data Types

R categorises data to know what mathematical operations or transformations are permitted. The four primary atomic vectors you will encounter in your careers as marine biologists:

1. **Numeric (numeric / double):** Continuous decimals (e.g., pH levels: 8.1, 7.95).
2. **Integer (integer):** Whole numbers (e.g., count of sea turtles spotted: 5L).
3. **Character (character):** Text strings enclosed in quotation marks (e.g., species name: "Acanthaster planci").
4. **Logical (logical):** Boolean values that are either TRUE or FALSE (e.g., Bleaching observed: TRUE).

Exercise: Checking Structures

R includes checking functions like class() and str() (structure) to verify your data categories:

```
# Assign variable values
site_name <- "Heron_Island"
transect_depth_m <- 12.5
bleaching_present <- TRUE

# Check using function class()
class(site_name)
class(transect_depth_m)
class(bleaching_present)

# Check using function str()
str(site_name)
str(transect_depth_m)
str(bleaching_present)
```

Note how R has automatically categorised these variables. Also note how the two functions return the same results, but in slightly different ways. One skill in programming is to know what function to use in different scenarios, and this will come with practice!

Exercise: rounding numbers

Let's try rounding an environmental tracking variable. Type out the following chunk exactly as written:

```
# Tracking the age of an old-growth Porites coral colony
years_old <- 25.765

# Clean this up for our summary report
round(years_olld, 2)
```

Task: Identify the spelling discrepancy. Fix the variable mismatch so that the code correctly

executes without throwing a missing object error.

Manipulating outputs

Let's see what happens when we assemble data text strings using the function paste(). Run this code block:

```
# Make variables
years_old <- 25.765
rounded_age <- round(years_old, 2)

# Combine text and data variables
paste("Average colony age is", rounded_age, "years old")
```

- **The Discrepancy:** Look at your printed output string. It reads "Average colony age is 25.76 years old". Now imagine the field protocol for this survey requires all data to be recorded at one decimal place. How could we achieve this?
- **Task:** Modify the arguments inside the round() function above so that the mathematical truncation matches the expected field report text exactly.

1.7 Data structures

Elements of data types can be combined to form a ***data structure.*** Think of elements as what you would put in a single cell in excel, with the whole spreadsheet the data structure.

R has many data structures. These include:

- `atomic vector`
- `list`
- `matrix`
- `data frame`
- `factors`

The ***vector*** is the most common, and is generally considered the workhorse of *R*. A vector is a **collection of elements** that are most commonly of the type `character, numeric, integer, logical`.

We actually already made a vector with numeric elements in it:

```
fish_lengths <- c(124, 152, 98, 221, 146)
```

But we can also make a vector full of character elements.

```
coral_spp <- c("Porites", "Acropora", "Montastrea")
```

Note all the elements inside a vector must be the *same type*.

Vectors

What happens when you interrogate the *type* of your character vector? Try `class(fish_lenths)` and the same for `coral_spp` . What happens if you try to make a vector of mixed variable types?

```
notes <- c("Acropora", 27.5, TRUE)
```

Lists

In R there are also ***lists***. Unlike vectors, values inside a list can be of several different types.

```
notes <- list("Acropora", 27.5, TRUE)
notes
```

The great thing about lists and vectors is that you can retrieve individual elements using double square brackets to reference their *index*.

```
notes[[1]]
```

Accessing elements will become really important later on, particularly if you want to pull out single values from a data frame or from the results of one of your statistical models.

There is lots to learn about structures that we don't cover in this workshop, so do read the section of the text book on data structure in our recommended readings.

Data frames and tibbles

In addition to data structures such as lists, R can also handle two-dimensional or "*rectangular"* data files. If you've ever used Excel or a similar program, you'll be familiar with spreadsheets. In R we refer to these as *"data frames"* and these are probably the most useful data structure for data analysis. They store data in much the same format as Excel, with rows and columns. Each column holds elements of the same *type* (meaning that data in each cell in a column of a dataframe need to be the same type).

An example of data frame, noting that all columns of a column are the same type:

1	"Plectropomus"	TRUE
2	"Scarus"	FALSE
3	"Pomacentrus"	TRUE
type: numeric	type: character	type: logical

You can import a data frame from an external file such as .csv or .xlsx file. You can also make a data frame in R by building one from the ground up. The function `data.frame` lets

you make them by adding together some different vectors (lists in this case) and giving each one a column name.

```
my_dataframe <- data.frame (no = c(1,2,3), c("Plectropomus", "Scarus",
"Pomacentrus"), c(TRUE, FALSE, TRUE)
my_dataframe
str (my_data_frame)
```

Notice that R has made a guess at the type of each variable. Sometimes it gets it wrong, so get into the habit of checking and changing it if necessary. You can do this by accessing the column and changing its type:

```
my_dataframe$no = as.factor(my_dataframe$no)
str (my_data_frame)
```

A factor is a categorical type, so here we are telling R that column 1 is actually storing a category (a group membership, say a numeric code for which team each of these people are in) rather than a real continuous number.

It's important that each vector is the same length (number of rows) because R won't allow you to have different columns with different lengths.

This is where you can use NA if you need something in a cell but have no data to go in it. Some others use an obvious number (e.g. 9999) but it's always better to use R's "Not Available" indicator because there are functions that can interpret it, such as `na.omit` (remove rows with NAs) or `is.na` (check if a value is not available).

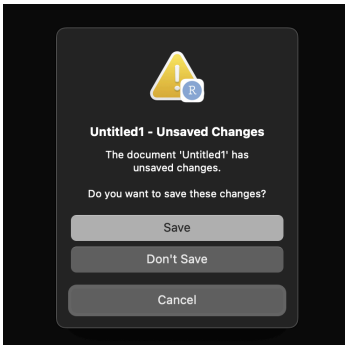
Despite being able to build data frames manually like this, you should avoid this type of practice for your own data. It can lead to transcription errors and is very time consuming (and boring!). Much better is to read your data into R using R's functions for importing data to allow you to store it as an object and start your analysis.

A tibble is a useful sub-category of data frame in R, which we will use extensively in this subject. For more information on what tibbles are see the help page here.

1.8 R Projects and workspace architecture

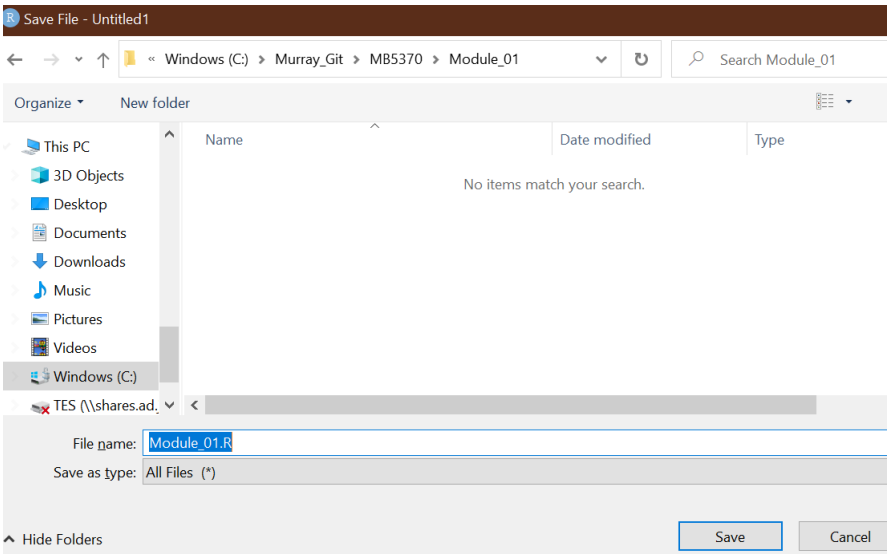
The working directory and saving your work

So, now you have got some code in your script you may have noticed that the title of the script has changed from "Untitled1" to "Untitled1*", meaning that RStudio has noticed that you've made a change to the script. Now if you try to close it down, it will give you a warning:



Great, we are now at the point where we want to save our new script! But where and how should we save and organise our R files!?! We'll get to some more detail on this in just a moment. For now, just save this script somewhere sensible on your computer and use the file menu or Ctrl+S to save your script. You can call your script `Module_01.R`.

As you go through the process of saving your script, you will notice that R suggests or "starts" its navigation at a top level directory. You might have seen something like this:



This is because R, much like your Windows explorer or Mac Finder has an "understanding" of the hierarchy of folders and files on your computer, and unless you tell it otherwise, it will usually default to the "top" level of your folder system. Commonly, and in this case, it was a drive called Windows (C:), and from there we could specify a "file path" down to where we wanted to save our script. Next we will learn a much more powerful way of managing our files and file paths.

R Projects: Escaping setwd() hell

One of the most destructive habits a data scientist can possess is hardcoding absolute file paths into their analysis scripts like this:

```
# Set working directory
setwd("C:/Users/DrShark/Desktop/MyDocuments/Class2026/Final_Data/Data/")
```

If you share a script with this line of code in it with a colleague, a tutor, or even open it on a different laptop, the code will instantly break because their directory architecture is different. You may have used setwd() in the past or even just clicked on the button inside R Studio, but that's about to change - **we never use setwd()**!!

Instead, we use **R Projects (.Rproj)**. An R Project acts as an anchor. When you open an .Rproj file, RStudio automatically shifts its workspace directory to whichever folder contains that file, allowing you to use safe, **relative file paths**, which will work on any computer.

Exercise: Getting set up with R Projects

Let's construct a bulletproof scientific repository architecture. This layout will serve as the template for your formal module assessments:

- In RStudio, navigate to the top menu and select **File > New Project**.
- Choose **New Directory > New Project**.
- Name your directory clearly: MB5370_Module_1_Coding. Save it somewhere logical on your machine.
- Look at your lower-right Files pane. You will see an active file named MB5370_Module_1_Coding.Rproj.
- Now, use the R Console (**not a script!**) to generate a clean, automated subfolder tree by running the following commands:

```
dir.create("data")
dir.create("code")
dir.create("output")
dir.create("docs")
```

Pause here, and have a think about what each of these folders are going to contain or be used for… Your directory tree should now look like this:

```
# MB5370_Module_1_Coding/
│
├── MB5370_Module_1_Coding.Rproj
├── data/       <- For raw, unedited data files (CSVs, shapes, etc.)
├── code/    <- For dirty/heavy data manipulation code
├── output/    <- For processed tables, figures, and saved objects
└── docs/       <- For your rendered write-ups and final documents
```

You already made a script for this module (Module_01.R) so go ahead and move that file into the code folder for this project.

1.9 Quarto for Reporting

While traditional R scripts (.R) are excellent for heavy, multi-step computation, they are poorly suited for sharing results with managers, stakeholders, or stakeholders who don't write code. Historically, scientists used R Markdown (.Rmd). Today, we pivot to **Quarto (.qmd)**, the open-source, next-generation standard for scientific and technical publishing.

Quarto uses **literate programming**. It weaves narrative prose (Markdown syntax), executable code blocks (in R or even in other programming languages), and high-quality rendering options into a single document.

Exercise: Launching Your First Quarto Document

1. Go to **File > New File > Quarto Document...**
2. Give it a sensible title and add your name as the Author. Select **HTML** as the output format and click **Create**.
3. Save this file immediately into your docs/ directory.
4. Look at the file structure. It is composed of three elements:
    - **The YAML Header (Top):** Surrounded by three dashes ---. This handles metadata, structural layouts, themes, and global formatting rules.
    - **Markdown Text:** Raw text with formatting shorthand (e.g., # Header 1, ## Header 2, *italics*, **bold**).
    - **Code Chunks:** Demarcated by three backticks and an R wrapper (```{r}). This is where your functional code executes.

Click the **Render** or **knit** button at the top of the script editor. Quarto will spin up a fresh, isolated background R session, run all the embedded code, convert your text markdown, and output a clean, standalone webpage.

1.10 Sourcing scripts from .Rmd/.qmd

As you transition to writing professional analysis pieces, you will quickly encounter the **Giant Code Dump Dilemma**. This happens when a researcher crams 500 lines of messy data cleaning, filtering, factor re-leveling, and error adjustments directly into the top of their beautiful clean report. It distracts from your scientific narrative and makes the document difficult to maintain.

Sourcing your code solves this completely:

1. You keep all your raw data cleaning inside a separate, robust script within scripts/.
2. You use the source() function to run that script invisibly from inside your polished Quarto document.

Exercise: Implementing a Clean Source Workflow

Here, we'll simulate go through the steps required to source your existing R script from your new .Rmd/.qmd file

1. Make sure your R script is saved in your project code folder
2. Open your qmd file and delete everything below the setup chunk

3. Replace it with this Quarto chunk:

````
```{r}
#| message: false
#| warning: false

# Execute workshop 1 R script
source("../code/Module_01.R") # change the filepath/name if required
```
````

4. Now run that chunk or hit **Render** or **knit** at the top.

You should see the output from your source code in your Rmd/qmd - neat! For the rest of this subject you should work primarily in Rmd or qmd, but feel free to utilise the source feature to help streamline your workflow.

1.11 Workshop review

We've covered a lot in this first workshop. Much of it may have covered things you're already familiar with. However, this module (and the activities we've done today) are the fundamental building blocks to a whole world of environmental computing that we'll cover in this subject.

Workshop 2 - Visualisation

Turning data into graphics with code

2.1 Workshop overview and schedule

Now you have a handle on the basic skills required to start coding in your marine science career, we will move onto some applied programming. This workshop aims to teach you how to visualise scientific information using R.

Data visualisation is a vital skill for all scientists that work with data and here we will use a dataset that is built-in to a package and explore how to make plots using the "grammar of graphics". Although the datasets we will analyse are fairly basic, working with straightforward data will allow you to master **ggplot2** and then begin to apply your learning to your own data. Trying out basic plotting methods with a simple, safe dataset can set you up with a strong foundation ready to move into your own, more complex analyses.

Table 3. Workshop 2 – Dataviz in R

Activities / Content	Notes	Links
Introduction	What have we learned? What's next? Principles of data visualisation (a discussion)	Slide Deck
Reading	What makes bad data visualisation?	
Introducing the tidyverse	How can I install the tidyverse	
Making your first plot	How do I read my data in R? How can I quickly check it out? How can I write my data?	
Understanding the grammar of graphics	What is the ggplot2 syntax?	
ggplot2 exercises	Learning the functions of ggplot2	
Communication with plots	How can I best communicate my results with ggplot2?	
Wrap-up	What have we learned? Any homework	
Finish		

This workshop will focus on working through the exercises in the R4DS 2e textbook, and you will gain an expert understanding of ggplot2 in the process. Although R has many methods for building plots and visualising data, by far the most popular is ggplot2. It's versatile, easy to use and, once you are used to the syntax, extremely flexible for supporting you to build any plot you wish (including maps!).

Objectives

Today we will focus on a few learning a few key fundamentals of ggplot2:

- Getting a dataset that is provided within the ggplot2 package solely for the purpose of comprehensively learning ggplot2.
- Learning the 'grammar of graphics' which will give you the strongest foundation of how to use the ggplot2 package for any purpose you wish.
- Understanding ggplot2 'aesthetics' so that you can quickly build plots the way that best conveys the message you want to spread with your plot
- Using ggplot2 to communicate your work to others
- Saving your plots developed in ggplot2
- Extending your skills in ggplot2 by applying them to your plot deconstruction exercise.

2.2 Reading about data visualization

Before starting the programming aspect of this module, please take 15-20 minutes to read the following chapter of Data Visualization (A practical introduction):

- What makes bad figures bad? https://socviz.co/lookatdata.html

Generally, you want to think clearly about the plots you plan to develop. What will best convey the message you want to convey? How can you be honest about your results? How can your plot be *professional*? Is it accessible to everyone? How can your plot help readers better understand your findings, as opposed to a description or table?

2.15 Working in your new project/repo

You should still be in your new R project in RStudio. Use your existing Rmd or qmd file in your or launch a new one in this project to continue your work for this workshop.

2.16 Load packages and data

First, you'll need to load the tidyverse.

```
library(tidyverse)
```

To provide the strongest foundation in ggplot2 possible, and fully understand its functionality, it's important to remove any component of data interpretation or uncertainty from the learning experience. We will therefore use the built-in dataset provided with ggplot2, which is designed *solely* to help you understand the functionality of the R package without being concerned about data issues, data wrangling (yet!) or even importing the dataset.

Why is using the built-in dataset important? Well, if you can disentangle your understanding of the ggplot2 'grammar of graphics', you'll build the strongest possible foundation that can then support your work to tackle any problem at hand.

The mpg data frame found in ggplot2 is a dataset with observations (234 rows of data) and variables (in the columns).

Have a look at the data frame by typing it into the R console.

```
mpg
```

The variables in the mpg data frame include:

- `displ`, which is the engine size of the cars in litres
- `hwy`, which is the car's fuel efficiency in miles per gallon (mpg). A car with a low fuel efficiency consumes more fuel than a car with a high fuel efficiency when they travel the same distance.

If you want to know more about this dataset, you can use `?mpg`.

2.17 Create your first ggplot

We want to create our first plot of this dataset. Using ggplot2, you can plot the data (`data = mpg`) and then put the size of the engine (`displ`) on the x axis and the fuel efficiency (`hwy`) on the y axis.

```
ggplot(data = mpg) +
  geom_point(mapping = aes(x = displ, y = hwy))
```

From the R for Data Science textbook (note: all indented text in this manual is directly from this textbook):

> The plot shows a negative relationship between engine size (displ) and fuel efficiency (hwy). In other words, cars with big engines use more fuel. What does this say about fuel efficiency and engine size?
>
> With ggplot2, you begin a plot with the function `ggplot()`:
>
> > `ggplot()` creates a coordinate system that you can add layers to. The first argument of `ggplot()` is the dataset to use in the graph. So `ggplot(data = mpg)` creates an empty graph, but it's not very interesting so I'm not going to show it here.
>
> You complete your graph by adding one or more layers to `ggplot()`:
>
> > The function `geom_point()` adds a layer of ***points*** to your plot, which creates a scatter plot. ggplot2 comes with many geom functions that each add a different type of layer to a plot. You'll learn a whole bunch of them throughout this chapter.
> >
> > Each **geom** function in ggplot2 takes a mapping argument. This defines how variables in your dataset are mapped to visual properties. The mapping argument is always paired with `aes()`, and the x and y arguments of `aes()` specify which variables to map to the x and y axes. ggplot2 looks for the mapped variables in the data argument, in this case, `mpg`.

2.18 Understand the ‘grammar of graphics’

2.18.1 Graphing template

One thing that can greatly help your ability to develop plots using ggplot2 is to use a template. In all cases, your ggplot code will follow a generic template like the one below. You will always call ggplot, offer it data, and then provide a **geom** function with a collection of mappings, which dictate how your plot will look.

Learn this template - it will help you realise exactly what you need to offer ggplot2 and how you can offer it, again supporting the development of your ‘foundations’ of data viz that you can slowly add complexity to as you develop more advanced visualisations.

```
ggplot(data = <DATA>) +
  <GEOM_FUNCTION>(mapping = aes(<MAPPINGS>))
```

So let’s revisit that. First, we make an empty ggplot() that simply provides the coordinate system that the remainder of the ggplot2 arguments can be mapped to. Try it.

```
ggplot()
```

What did that do? Yes, it simply created your plot window, but without the data argument the plot window has not yet applied any axis limits.

```
ggplot(data = mpg)
```

2.18.2 Aesthetic mappings

In addition to simply plotting coordinates of x, y data, you can add a third variable, such as class, by mapping it to an **aesthetic** represented as `aes()` in your template from above.

What is an aesthetic? Well, this is a visual property of the objects you’ve already plotted, and can include things like:

- size
- shape
- colour

So when you’re creating a plot, you essentially need two attributes of a plot: a **geom** and **aesthetics**.

Try the following exercises to explore the aesthetic mapping available to you.

> You can convey information about your data by mapping the aesthetics in your plot to the variables in your dataset. For example, you can map the colors of your points to the class variable to reveal the class of each car.
>
> To map an aesthetic to a variable, associate the name of the aesthetic to the name of the variable inside `aes()`. ggplot2 will automatically assign a unique level of the aesthetic (here a unique color) to each unique value of the variable.

Change point colour by class.

```
ggplot(data = mpg) +
  geom_point(mapping = aes(x = displ, y = hwy, colour = class))
```

Note above we mapped class to colour, but you can map to the size aesthetic in exactly the same way. Note how the sizes of the points now represent the class of car. Also note how ggplot2 gives you at least a little bit of guidance about what makes good data visualisation (ie. don't plot a variable which is not continuous to a continuous aesthetic like size).

Change point size by class.

```
ggplot(data = mpg) +
  geom_point(mapping = aes(x = displ, y = hwy, size = class))
#> Warning: Using size for a discrete variable is not advised.
```

Change point transparency by class (alpha). Here the alpha aesthetic is what dictates the transparency. Also consider trying a simple number, like 0.3 to this aesthetic, which globally applies a transparency to all points. This is very helpful for dense point clouds, where you want a reader to see that there are overlapping data points. See the ggplot2 alpha reference material for a few examples of the use of alpha.

```
ggplot(data = mpg) +
  geom_point(mapping = aes(x = displ, y = hwy, alpha = class))
```

Change point shape by class.

```
ggplot(data = mpg) +
  geom_point(mapping = aes(x = displ, y = hwy, shape = class))
```

You can also set these properties manually, such as by offering a number or a colour. Let's make all of our points blue.

```
ggplot(data = mpg) +
  geom_point(mapping = aes(x = displ, y = hwy), color = "blue")
```

In this example, it simply changes the appearance of the plot, it doesn't show anything about the nature of the variable you've plotted. That's fine, but make sure it makes sense from a data viz point of view.

There are a range of manual aesthetics you can set to manually alter the appearance of your plot. These include:

- The name of a color as a character string.
- The size of a point in mm.
- The shape of a point as a number, as shown in Figure 3.1.

A question to try out:

1. What happens if you map an aesthetic to something other than a variable name, like `aes(colour = displ < 5)`? Note, you'll also need to specify x and y.

2.19 Troubleshooting

As with all programming you'll run into some problems. But that's OK! It's very common for data scientists or analysts to have to keep testing and trying (no one gets it first go!).

As in module 1, **from this point forward I will add in some minor errors to the code here to avoid you simply cutting and pasting this code**. The idea is that you learn to read the code and debug your problems.

Despite this, there will be some other common problems you're likely to face. Perhaps the main one with ggplot is having the + in the wrong place. It has to go at the end of the line (not the start of the next line), which is particularly important if you're running your script line-by-line.

Example:

```
ggplot(data = mpg)
+ geom_point(mapping = aes(x = displ, y = hwy))
# the + should be on top line
```

As with all programming, there are some really useful methods for debugging your code. Revisit the Module 1 workshop where we discussed using online help, reading the R help pages, learning to read your own code carefully and precisely, and trying to interpret the error messages you've received.

As a very last resort, you can always ask the instructor (who will probably just tell you to do the above anyway).

2.20 Facet and panel plots

If you've read any scientific literature with figures, you'll note that a common practice is to break a single complex plot into many sub plots (or panels). This allows you to develop separate plots for a range of reasons, most often to show a **subset** of your data.

In ggplot2, you'll typically do this using **facets.**

To facet your plot using a single variable, use `facet_wrap()`. Facet wrap syntax is in the function of a formula (kind of like a linear model formula), where the `~` dictates which variable you want to subset your data with.

Note: only use `facet_wrap()` for discrete variables.

```
ggplot(data = mpg) +
  geom_point(mapping = aes(x = displ, y = hwy)) +
  facet_wrap(~ class, nrow = 2)
```

If you want to do this with more than one variable, then use `facet_grid().` Here you need two variables using `~` to split them up.

Facet plot on the combination of two variables using `facet_grid()`. Facet grid needs two variables separated by a `~`.

```
ggplot(data = mpg) +
  geom_point(mapping = aes(x = displ, y = hwy)) +
  facet_grid(drv ~ cyl)
```

Use a `.` if you do not want to facet in the rows or column dimension. Note what happens in this case:

```
... +
facet_grid(. ~ cyl)
```

Exercise:

1. Read ?facet_wrap. What does nrow do? What does ncol do? What other options control the layout of the individual panels?

You'll note that with a tool like this, you can get very close to building publication quality plots directly in ggplot2, at a quality where you probably don't need to do any formatting in Microsoft Word to handle the panelling problem.

2.21 Fitting simple lines

An outstanding feature of ggplot2 is that you can use a variety of visual objects to represent your data. So far we have used points that are plotted at the coordinate location of the x and y axes for each data point.

To display data as points, use `geom_point()`.

```
ggplot(data = mpg) +
  geom_point(mapping = aes(x = displx, y = hwy))
```

However, ggplot2 can use a variety of geom objects to represent the data. Here, we might want to use bar plots, line charts, boxplots and so on. Well we can handle this issue in ggplot directly using a different geom to plot the same data. Here, instead of plotting points, we will use a smooth line.

To display the same data as a smooth line fit through the points use `geom_smooth()`.

```
ggplot(data = mpg) +
  geom_smooth(mapping = aes(x = displ, y = hwy))
```

So let's recap. A **geom** is an object that your plot uses to *represent the data*. To change the geom type in your plot, simply change the geom function that you add to your plot template. Sometimes you may want to try a few things out, in which case you could use comments to help you remember what worked and what didn't.

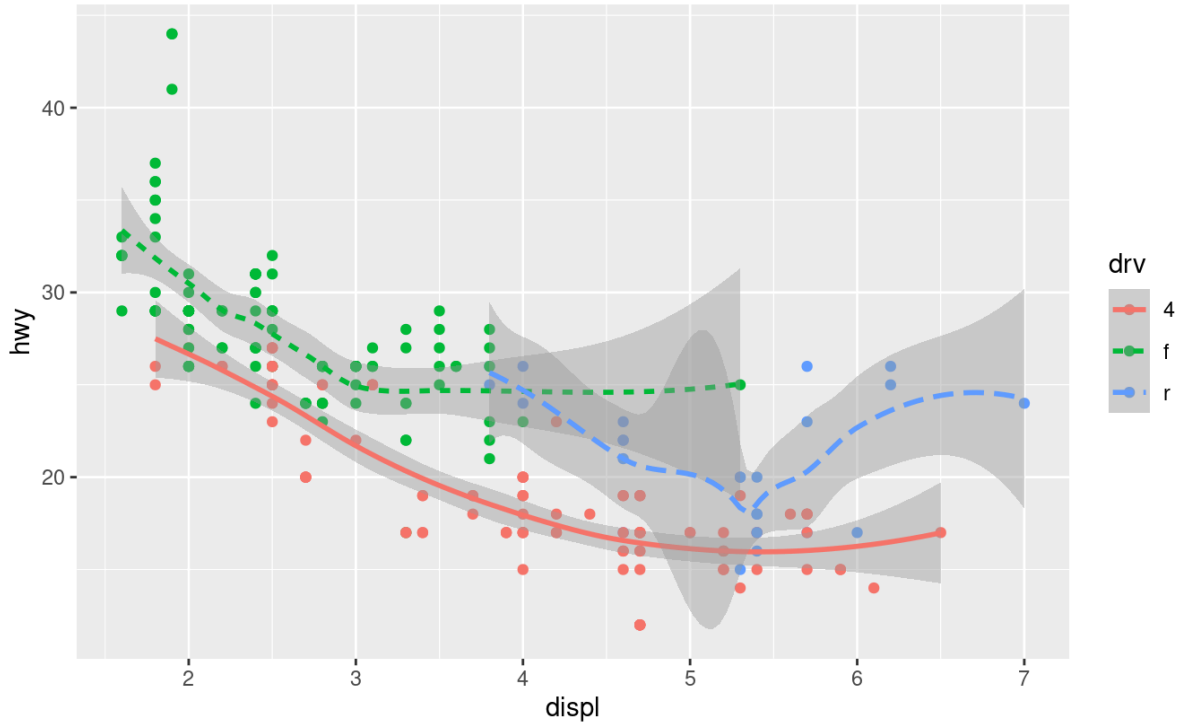
```
ggplot(data = mpg) +
#  geom_point(mapping = aes(x = displ, y = hwy)) # points horrible
```

```
  geom_smooth(mappings = aes(x = displ, y = hwy)) # try smooth
line
```

You can also change the line type, and like we did with colours, you can use a variable to control it. Note: here it doesn't make much sense, but you can see we have different types of dashes when we use the linetype argument with `geom_smooth()`, which separate the cars into three lines based on their `drv` (front wheel, rear wheel or 4wd) value.

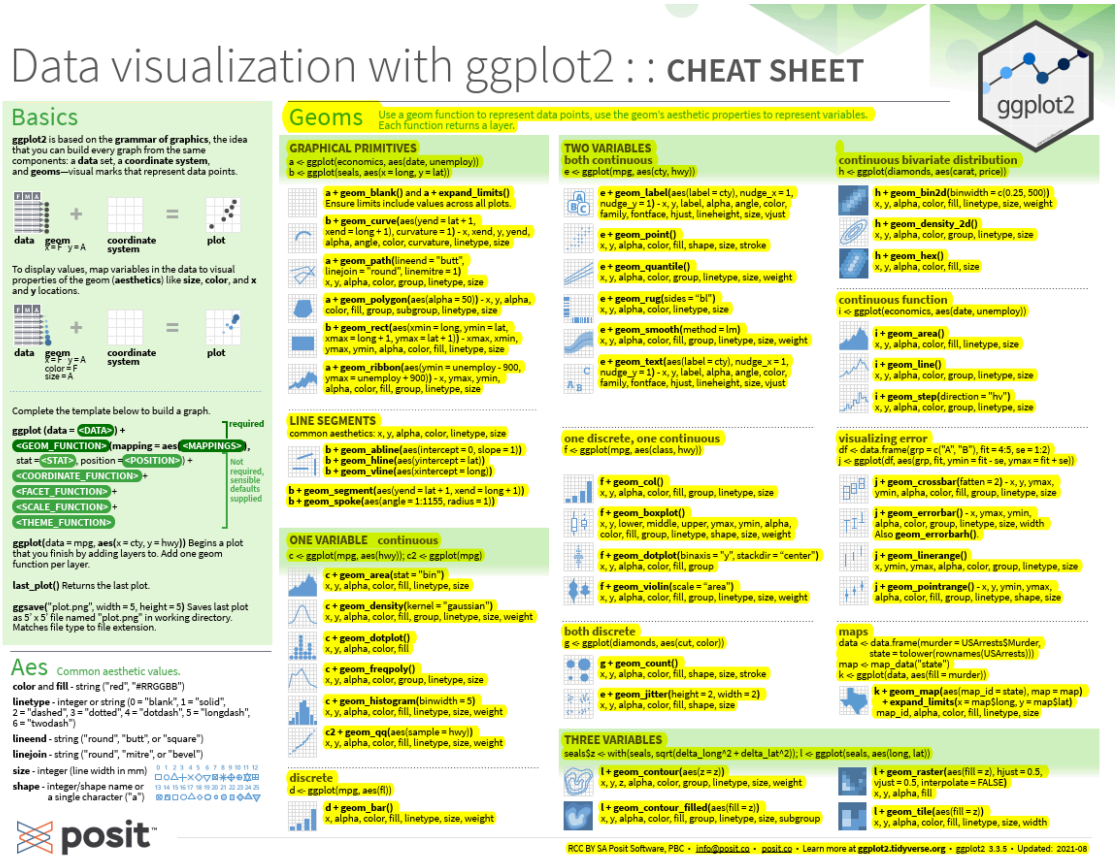
```
ggplot(data = mpg) +
  geom_smooth(mapping = aes(x = displ, y = hwy, linetype = drv))
```

It might be hard to to understand what's going on, but here you can see we are fitting a smooth line to different subsets of the data.



Here we have two **geoms** on the same graph (points *and* lines)!

ggplot2 has over 40 different **geoms**! Have a look at the ggplot2 cheatsheet for some examples - all of the below in yellow are different **geoms** you can use for various things.



This is getting fairly exciting and shows how flexible ggplot2 can be, but let's lock in some ideas before moving onto making our own plots with multiple geoms.

So far we've only made fairly simple plots. But what happens if you want to group objects by a categorical variable (like species or sex or site)?

Here we will use the `group` argument to show that our data is in fact grouped. It won't add a new legend but can easily be used to vary your plots by the discrete variable.

Set the `group` aesthetic to a categorical variable to draw multiple objects.

```
ggplot(data = mpg) +
  geom_smooth(mapping = aes(x = displ, y = hwy, group = drv))
```

Change the color of each line based on `drv` value

```
ggplot(data = mpg) +
  geom_smooth(
    mapping = aes(x = displ, y = hwy, color = drv),
    show.legend = FALSE,
  )
```

Now, let's increase our complexity even more. Here we will plot **multiple geoms** on the single plot. All you need to do is to add them together. This one is nice for showing the underlying data and how it relates to the `geom_smooth` line.

```
ggplot(data = mpg) +
  geom_point(mapping = aes(xx = displ, y = hwy)) +
  geom_smooth(mapping = aes(x = displ, y = hwy))
```

This is often exactly what we want to do! However, note how the top and second rows are *duplicated*, meaning if you wanted to change the x variable in your plot, you'd need to change it in several locations!

This is not ideal in a programming sense, and can increase the chance you'll make an error. Therefore, ggplot allows you to pass these 'overarching' mappings to the `ggplot()` argument, making them *global* mappings that are applied to every single subsequent **geom**. Much in the same way that the data argument is also *global*, it is used every time a geom is called.

Here's an example of where we make the exact same plot as above, but it is programmatically more efficient and makes it easy for you to change a variable that you want to plot on the x or y axis.

```
ggplot(data = mpg, mapping = aes(x = displ, y = hwy)) +
  geom_point() +
  geom_smooth()
```

The extra nice thing is that you can still use mappings under `ggplot()` to reduce duplications in code (like in the above code). This is useful if you want to change point styles or develop your own customisations.

Use mappings in specific layers to enable the display of different aesthetics in different layers (super handy!). Note how the line is not styled by class, but you can style the points by themselves.

```
ggplot(data = mpg, mapping = aes(x = displ, y = hwy)) +
  geom_point(mapping = aes(color = class)) +
  geom_smooth()
```

The same goes if you want to specify different data for each layer. Here, we use a filter (`class = "subcompact"`) to select a *subset* of the data and *plot only that subset*.

```
ggplot(data = mpg, mapping = aes(x = displ, y = hwy)) +
  geom_point(mapping = aes(color = class)) +
  geom_smooth(data = filter(mpg, class == "subcompact"), se = FALSE)
```

You will learn what exactly `filter()` does later on in this module, but for now just note how it only shows a small subset of the dataset.

Exercise:

1. What geom would you use to draw a line chart? A boxplot? A histogram? An area chart?
2. Run this code in your head and predict what the output will look like. Then, run the code in R and check your predictions. Will these two graphs look different? Why/why not?

```
ggplot(data = mpg, mapping = aes(x = displ, y = hwy)) +
  geom_point() +
  geom_smooth()

ggplot() +
  geom_point(data = mpg, mapping = aes(x = displ, y = hwy)) +
  geom_smooth(data = mpg, mapping = aes(x = displ, y = hwy))
```

2.22 Transformations and Stats

We are going to learn about easy transformations and data summaries using another dataset provided in ggplot2, specifically to support the teaching of transformations. The `diamonds` dataset comes in ggplot2 and contains information about ~54,000 diamonds, including the `price, carat, color, clarity`, and `cut` of each diamond.

2.22.1 Plotting statistics

Our first bar chart shows that more diamonds are available with high quality cuts than low quality cuts.

```
ggplot(data = diamonds) +
  geom_bar(mapping = aes(xx = cut))
```

From our textbook:

> On the x-axis, the chart displays cut, a variable from the `diamonds` dataset. On the y-axis, it displays count, but count is not a variable in `diamonds`! Where does count come from? Many graphs, like scatterplots, plot the raw values of your dataset. Other graphs, like bar charts, calculate new values to plot:
>
> - *bar charts*, *histograms*, and *frequency polygons* "bin" your data and then plot bin counts (the number of points that fall in each bin).
> - *smoothers* fit a model to your data and then plot predictions from the model.
> - *boxplots* compute a robust summary of the distribution and then display a specially formatted box.
>
> The algorithm used to calculate new values for a graph is called a **stat**, short for statistical transformation.

This incredible functionality of ggplot2 will open up a wealth of ways you can visualise your data, without the need to fit statistical models or summarise your dataset! It's hard to describe how time-saving this can be, especially since this feature is as flexible as the rest of ggplot2.

You can generally use **geoms** and **stats** interchangeably. For example, you can recreate the previous plot using stat_count() instead of geom_bar().

```
ggplot(data = diamonds) +
  stat_count(mapping = aes(x = cut))
```

The main reason this is so easy is because *every geom has a default stat* and *every stat has a default geom*, which means you don't need to worry about what's going on.

2.22.2 Overriding defaults in ggplot2

Just like in general R and even ArcGIS, it's important to understand that the *defaults are not the only thing you can do*. However, the fact that defaults are there in the first place can have big implications for your results, so you should always make an effort to understand any of these 'black box' outputs.

What is the default? How can it change? What's it doing to my work?

You might want to *override* a default stat now that you understand what the defaults are. Change the default stat (which is a count, a summary) to identity (which is the raw value of a variable).

Don't worry yet about the "tribble()" function from the code below. It's just used to make a tibble and we will learn about those in a bit (they're basically a dataframe).

```
demo <- tribble(
  ~cut,         ~freq,
  "Fair",       1610,
  "Good",       4906,
  "Very Good",  12082,
  "Premium",    13791,
  "Ideal",      21551
)
demo

ggplot(data = demo) +
  geom_bar(mapping = aes(x = cut, y = freq), stat = "identity")
```

You can also **override** a default mapping from transformed variables to aesthetics. For instance, you could display a bar chart of the *proportion* of your total diamond dataset rather than a count.

```
ggplot(data = diamonds) +
  geom_bar(mapping = aes(x = cut, y = stat(prop), group = 1))
#> Warning: `stat(prop)` was deprecated in ggplot2 3.4.0.
```

2.22.3 Plotting statistical details

You might also want to show a little more about these transformations in your plot, which is good practice to be transparent about uncertainty or any other limitation of your data.

You can do this using `stat_summary()`.

```
ggplot(data = diamonds) +
  stat_summary(
    mapping = aes(x = cut, y = depth),
    fun.min = min,
    fun.max = max,
    fun = median
  )
```

2.23 Aesthetic adjustments

Another way to boost the way you can convey information with plots using ggplot2 is to use **aesthetics** like `colour` or `fill` to change aspects of bar colours.

```
ggplot(data = diamonds) +
  geom_bar(mapping = aes(x = cut, colour = cut))
ggplot(data = diamonds) +
  geom_bar(mapping = aes(x = cut, fill = cut))
```

Now try using these aesthetics to colour by another variable like `clarity`. Notice how the stacking is done automatically. This is done behind the scenes with a **position** argument.

```
ggplot(data = diamonds) +
  geom_bar(mapping = aes(x = cut, fill = clarity))
```

The ability to make **position adjustments** is vital, it allows you to customise your plots in three ways, `identity` (raw data), `fill` (changes heights) and `dodge` (which forces ggplot2 to not put things on top of each other)

- If you use `position = "identity"`, you will be able to place each object exactly where it falls in the context of the graph. This is vital for point charts like scatter plots but makes a mess in a bar plot situation by showing too much information (a bar plot generally summarises information). So in this case we will need to alter the bar aesthetic.

```
#To alter transparency (alpha)
ggplot(data = diamonds, mapping = aes(x = cut, fill = clarity)) +
  geom_bar(alpha = 1/5, position = "identity")

#To color the bar outlines with no fill color
ggplot(data = diamonds, mapping = aes(x = cut, colour = clarity)) +
  geom_bar(fill = NA, position = "identity")
```

- `position = "fill"` works like stacking, but makes each set of stacked bars the same height.

```
ggplot(data = diamonds) +
  geom_bar(mapping = aes(x = cut, fill = clarity), position = "fill")
```

- `position = "dodge"` places overlapping objects directly *beside* one another.

```
ggplot(data = diamonds) +
  geom_bar(mapping = aes(x = cut, fill = clarity), position = "dodge")
```

One bonus position adjustment is `jitter`, which slightly moves points so you can see them all (especially when they overlap). I'm sure you've seen really nice box plots with jittered points all over them, which you handle with this function.

- `position = "jitter"` adds a small amount of random noise to each point to avoid overplotting when points overlap. This is useful for scatterplots but not barplots.

```
ggplot(data = mpg) +
  geom_point(mapping = aes(x = displ, y = hwy), position = "jitter")
```

2.24 The layered grammar of graphics

The sections above have given you the foundation to make almost any type of plot you want to in ggplot2.

It's good now to update our template for making a ggplot2. Here it is, with position adjustments, stats, and faceting (recall how facets made us panel plots of subsets?).

Later in this module we'll learn about coordinates, which allow for mapping, but for now let's keep it this simple. Most of the time you won't need to populate all of these, because ggplot2 has defaults for most of them, but overall you have enough here to build almost any plot.

```
ggplot(data = <DATA>) +
  <GEOM_FUNCTION>(
     mapping = aes(<MAPPINGS>),
     stat = <STAT>,
     position = <POSITION>
  ) +
  <FACET_FUNCTION>
```

To read more about this 'grammar of graphics', go to page 34 of our textbook that explains how by offering information on these key aspects of a graph, you can build any plot you wish.

Workshop 3 - Reproducible research in marine science

3.1 Session schedule

This workshop will build on our base of scientific programming skills in the context of making our data analysis transparent and reproducible, all while keeping on top of versions of our work as it progresses..

In particular, it focuses on formalising your ability to build rigorous programmed workflows and begin to put into practice methods for backing-up your code and implementing version control, which will support your ability to work collaboratively and strive towards open science principles.

This workshop session will introduce you to Git, GitHub, R markdown and RStudio projects. It aims to set us up to write code in an organised way, using good, consistent style and developing modular and well-commented scripts.

Table 5. Introduction to version control

Activities / Content	Notes	Links
Subject introduction Workshop introduction What we will achieve Open discussion Questions	What will we achieve in this module? What homework will I need to do? What is this workshop about? How can version control help me?	Install R Install Git Register GitHub account
Automated version control	What is version control and why should I use it?	
Setting up Git and Github	Git & GitHub setup	
Create a repository on github	How can I use Github to save my work?	
Local to remote sync (Rstudio to Github)	What is the process for syncing my work to Github?	
Assignment	Continue on the first of two assignments for your e-portfolio	
Finish		

3.2 Session overview

In this workshop session we will learn about version control using Git and GitHub.

Why should you use version control? Well, it can help you save time and ensure safe backing up, as well as helping your future self (thank me later). Using a version control system like Git is a critical component in building reproducible and open access programming pipelines, and this workshop aims to get you up and running with some practical experience in its use.

You will submit a link to your GitHub profile as Assessment Item 3 for this subject.

As noted in the lecture, Git is designed to track changes to your files. It stores your files remotely, using GitHub in our case, and will enable you to collaborate with others (and your future self!) and incrementally and carefully develop your scripts.

Today's session primarily focuses on interfacing with GitHub from our local machines, using RStudio. Do note you can work with Git in a range of ways, including via the command line. GitHub's desktop app, or the tools that Git itself provides (like Git BASH). However, for the

majority of you we will simply use Git for coding in R to backup and version our scripted workflows and analyses, so RStudio will provide you the basis you need.

If you want to know much, much more about Git, then do read the Happy Git and GitHub with R website. Much of these materials below benefit from the Happy Git and GitHub with R website and have been adapted from the Ocean Health Index open-source training material.

Objectives

Today we will access GitHub from our local machines using RStudio. Here's what we'll do:

- Clone the module workshop repository from GitHub
- Use the embedding scripts to work through the DataViz session
- Make a new repository on our machines and sync to GitHub using RStudio
- Learn the RStudio to Github workflow that enables version control (including commit, push and pull)
- Explore our files on Github
- Practice by making changes to our scripts
- Set up a markdown document for this module
- Begin our project for the module.

3.3 Git and GitHub

Avoiding poor practices of versioning your work (e.g. final1.R, final_final1.R, better_final_final1.R) will help you become a better, more efficient and highly collaborative marine biologist and avoid the trauma of not remembering which is your final R script.

When you access your scripts via a repository, you will always see your most recent version. Even better, you can quickly compare your scripts with previous versions and revert back to them if you need to.

In addition:

1. You can work with others at the same time on a script. It's hard to imagine now why you would ever want to do this, but when the time comes it will be good to know that there's software specifically designed to help you do this.
2. Easy to share your files, figures, outputs and progress via the GitHub website.
3. Git is a second-to-none backup system, all for free!
4. You can access your analyses on any computer at any time anywhere, all you will need is an internet connection.
5. Use of version control signals that you're a skilled marine data scientist that understands why good practices in data and code management matter.

Now, remember from the lectures that **Git** and **GitHub** are two differing things:

- **Git** is a version control system that can track any file type, though simple and small text work best (like .txt, .csv, .r)
- **GitHub** is a website that handles the storage of your versioned files. It's especially nice for visualising file changes and sharing data.

GitHub is owned by Microsoft and can do many things, but for us we'll just use its ability to store our version controlled repositories in a public place.

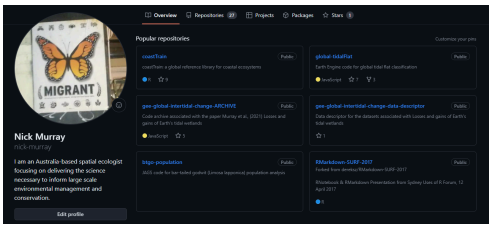
Terminology

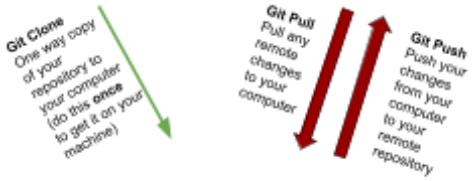
You'll hear many terms in this workshop, but the main ones are below. Discuss with your colleagues and see if you can come up with definitions for these terms.

- Repository
- Local
- Remote
- Stage
- Commit
- Push
- Pull
- Clone

Note there are a huge amount of commands you can use in Git and Github, but these are mostly designed for professional software developers. As marine biologists, we're going to use only a few (e.g. Commit, Push, Pull) 99.9% of the time.

GitHub: a website for storing file repositories (folders) under version control





3.4 Setting up Git and GitHub

We installed R and RStudio in Workshop 1, so you shouldn't need to do that again, but if you want to check your version(s) and make sure everything is up to date, follow the instructions here.

Git

Next, we need to set up Git on our computers, so that we can implement version control locally. Remember that Git is a program that sits on your computer, whereas GitHub is the public face and the way to share your code. You can use Git without GitHub to track files locally. Git is already installed on JCU computer lab machines, but to set up on your computer you need to install Git and get it talking to your machine. Carefully follow the instructions in Section 6 and 7 of Happy Git. Make sure to follow the instructions specific to your operating system!

GitHub

Once you have Git up and running on your machine it's time to set up a GitHub account! Follow the instructions here. For tips on selecting a username check out this advice.

GitHub has implemented mandatory two-factor authentication (2FA) so we also need to set that up now. There are a number of options available for setting up 2FA, but GitHub actually has a phone app that you can install and use for this, which can be quite convenient.

You should now have Git set up on your computer and a GitHub account. Congratulations! Now we need to get Git and GitHub talking to each other. Carefully follow these instructions to do this, and note you can use either a special package in R (*usethis*) or do it directly in the terminal or bash on your computer.

If you have any **problems**, please see this troubleshooting section in the HappyGitWithR website.

For those with **MacBooks**, see the guidance here.

Before we move on, we need to check everything is talking to each other. There are heaps of ways to do this, but for now let's use the credentials package or the usethis package.

To install usethis make sure to use the console (not a script or qmd!)

```
install.packages("usethis")
```

```
credentials::git_credential_ask()
usethis::git_sitrep()
```

Configure a Personal Access Token

In addition to 2FA, GitHub also requires configuration of a Personal Access Token (PAT) which is the actual credentials that are used to verify your access to your repos. For more information on GitHub PATs check out the GitHub help pages. To set up your first PAT, head to this guide and follow the steps. Note the recommendation that you use a PAT that expires, but if you don't want to have to keep creating a new PAT then select an expiration option of "No expiration".

Excellent! You are now ready for a life of reproducible data analysis and version control. Congratulations!

3.5 Using the class GitHub repository

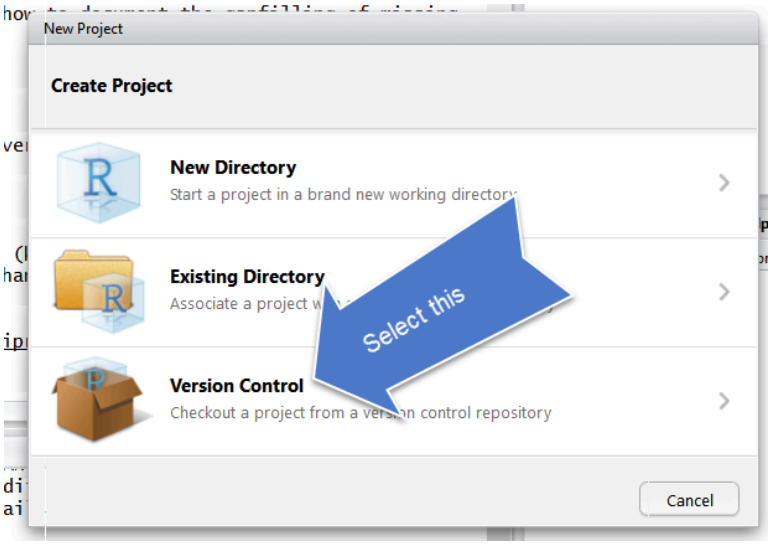
Now you are set up with Git and GitHub the next task is to organise your work in this module so far into a nice organised project and push it to your GitHub profile. There is an example of how a project can be organised in a repo available on the class GitHub organisation. You can navigate to this directly in your browser and view there. Or if you wish you can clone a copy to your own computer and copy/paste files and folders as required.

The next steps will take you through getting the class repo on to your own computers, but we have hidden a few deliberate coding "errors" in there. These are designed to build your eye for detail in your coding and develop your troubleshooting skills, so keep a look-out!!

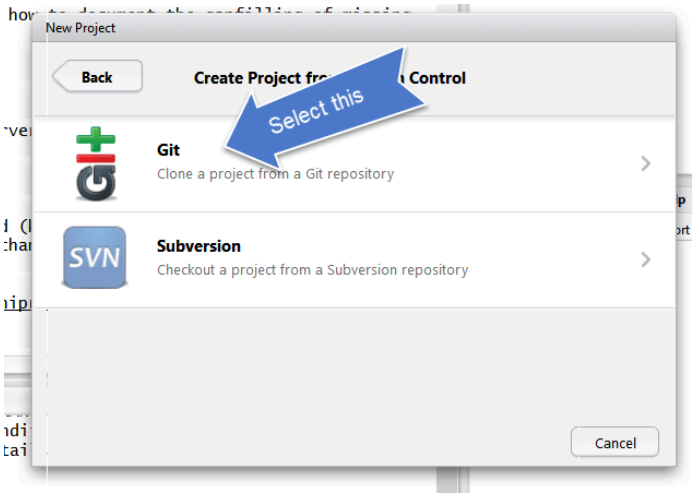
Clone the class repo to your computer

You can view any repo on GitHub and the code within it, as long as it's public. The class repo can be found here. Follow these steps to clone a copy of the class repo to your computer:

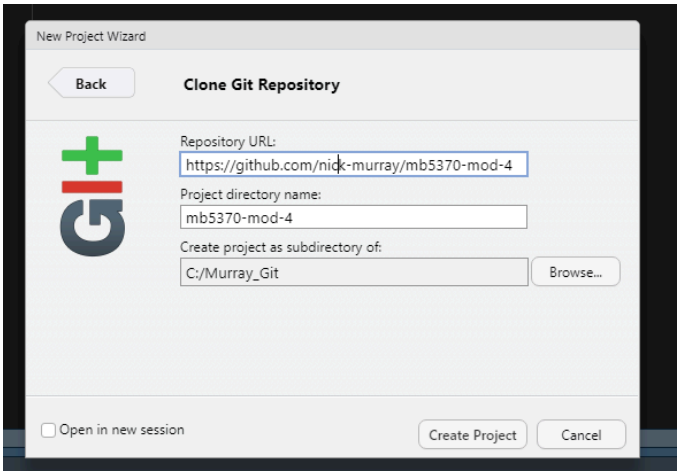
1. First make sure you have a suitable folder where you can (and will!) store all of your data analysis work, now and in the future. Give it an appropriate name (e.g. "github" or "data_analysis") and make sure it is somewhere sensible on your computer.
2. Copy the web address of the repo you want to clone.
3. Now, in R Studio you need to make a new project. File -> New Project
4. When you create your new project, click Version Control



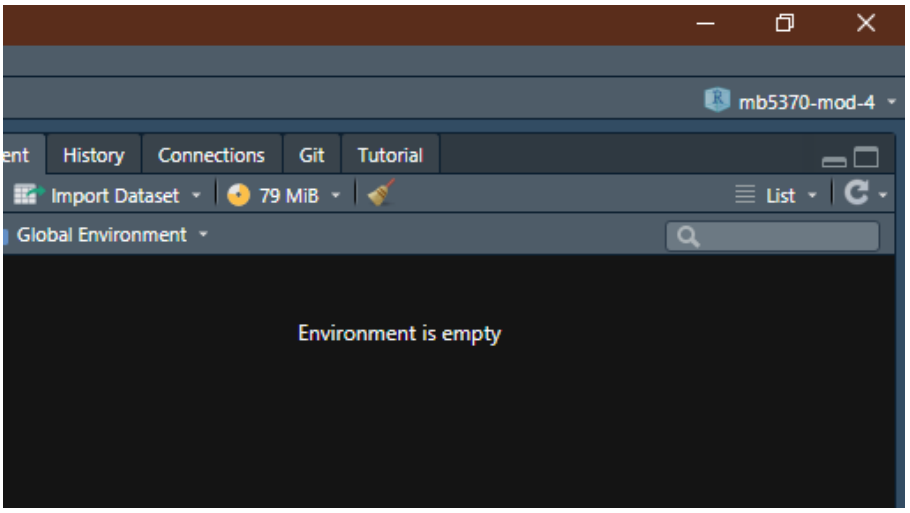
5. Now, select **Git**



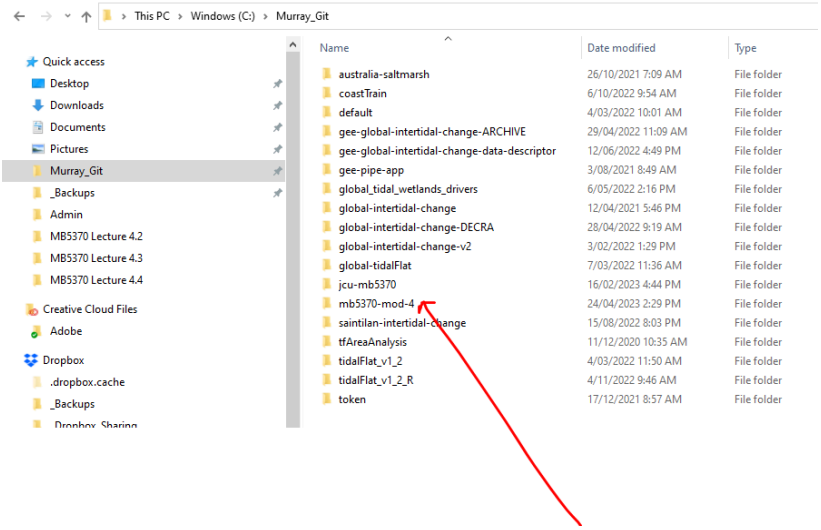
6. In the next pop-up window, paste in the url for the repository that you just copied, check the project directory name has auto-populated to your repository name.
7. In the ‘create … as subdirectory…’ section put in the path to your ‘github’ folder that you just made.



Now, if it all went to plan you will have the repo name in the project section of RStudio, right up in the top right.



And… if you check your local drive (e.g. Windows File Explorer or MacOS Finder) you should find your repository and its contents in your folder.



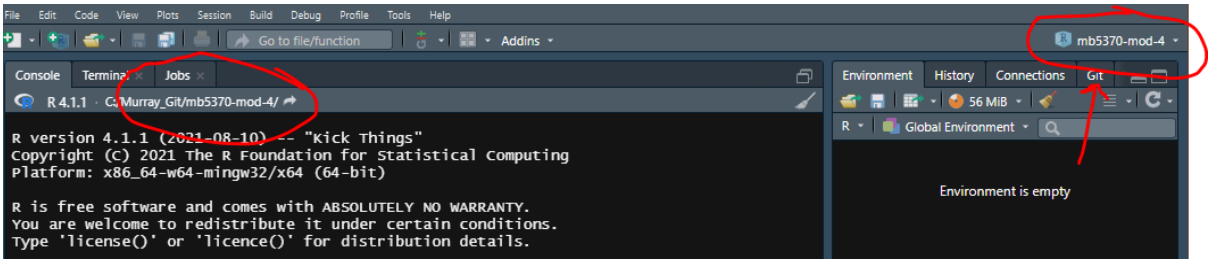
Review the class repo on your machine

There are a few things to notice now that you've *cloned* a repository (which is a version controlled folder) to your local computer. These are:

1. RStudio has automatically set your working directory to your repository. Now you can happily save files here or export them, without needing to write the full file path to the folder (like we normally have to).

2. Rstudio has added an .RProj file to your folder. This file simply helps RStudio manage your R *project*.
3. We now have a Git tab. This tab allows you to interface to GitHub via RStudio.

GitHub is now responsible for everything that happens in your working folder (which is your working directory in R, and a GitHub *repository*).

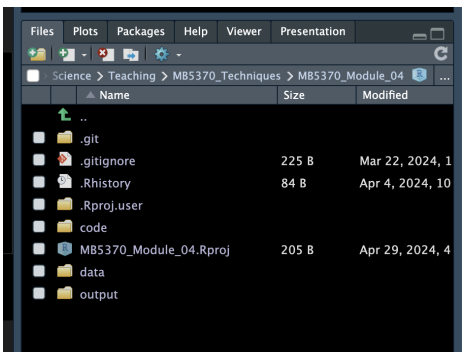


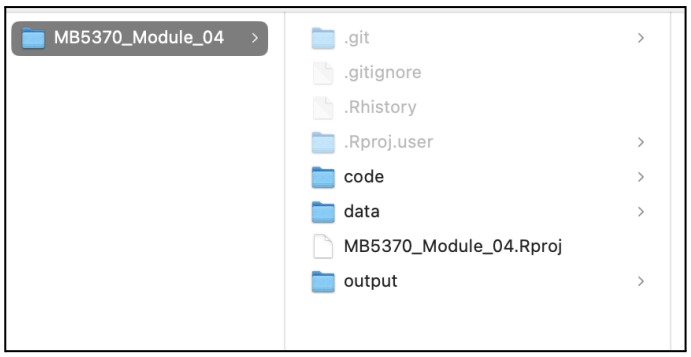
A note on workflow and project structure

There are lots of ways to structure your projects and manage your coding work, but the important thing is to put some time into thinking about how you are going to manage this for your own projects. You will notice that the class repo that you just cloned has the same simple folder structure as we discussed earlier:

```
# MB5370-Programming/
│
├── MB5370-Programming.Rproj
├── data/       <- For raw, unedited data files (CSVs, shapes, etc.)
├── code/       <- For dirty/heavy data manipulation code
├── output/     <- For processed tables, figures, and saved objects
└── docs/       <- For your rendered write-ups and final documents
```

You can see this structure either in your R studio files pane or in your file manager app:





There are other folders that you may wish to include, or different structures that you'd like to use, depending on the kind of analysis or project you are working on. This blog discusses this topic in some detail and more guidance by two outstanding Australian ecologists will give you some ideas on the folders you should add to every project.

3.6 Track your own project

So, you've cloned a repo from GitHub to your local machine. Excellent! Now it's time to start tracking your own project and push it to GitHub. To do this you have three main options. For all three, make sure you are in RStudio, and in your project.

Using usethis

In the R Console, call usethis::use_git().

Using tools

In RStudio, go to Tools > Project Options … > Git/SVN. Under "Version control system", select "Git". Confirm New Git Repository? Yes!

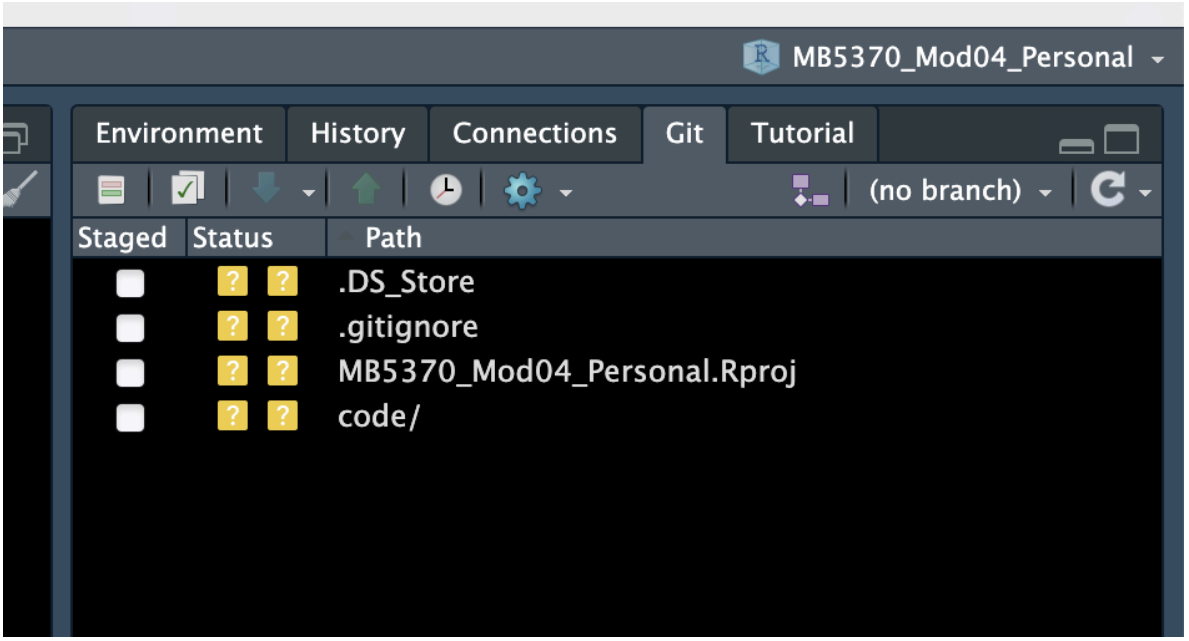
Using the terminal

Click on the terminal pane at the bottom. With working directory set to the project's directory, run git init.

You now have a new project on your computer that is being tracked by Git. Excellent!! Now when you do *anything* in your repository on your computer, such as saving an R script in your **R** folder, pasting a .csv data file into the **data** folder, or writing a report or assignment in the **output** folder, R will tell you that you've done something to your repo in the Git tab.

To demonstrate this, in your file explorer app (Windows Explorer or Mac Finder), copy and paste the three folders from the class repo to the new project you just created (make sure you don't copy across the .Rproj file - this file is unique to each project!)

Your Git tab should now look something like this:



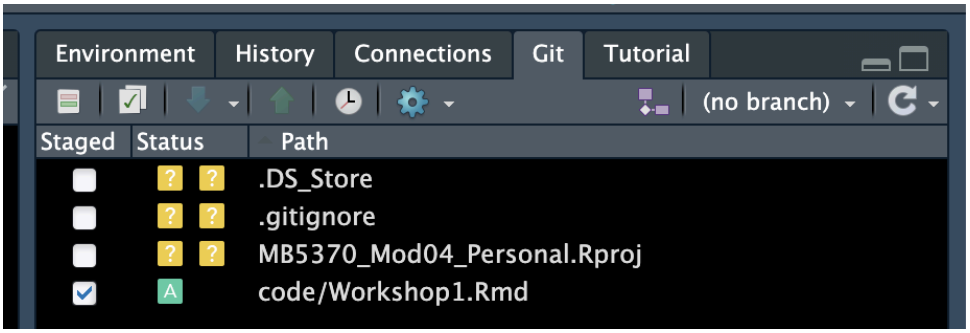
You will notice a few things about this:

1. Question marks next to each of the files that Git has "noticed".
2. That Git has "noticed" two files called .DS_Store and .gitignore, which we didn't add?

Also note that if you create a folder with nothing in it (yet), then Git will ignore it until there is a file in there.

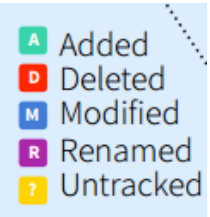
We'll cover these little quirks now.

1. In the Git tab, R studio uses coloured boxes to tell you what you've done to your repo (legend to the boxes is below). The question mark means that, although Git has noticed the file, it's not being tracked. Try clicking the check-box on the "code/" line:



You'll see that the "**?**" changes to an "**A**", because now it has been added to the files that you want Git to track. This is known as staging and we'll be doing plenty of this in the rest of the workshops. If you delete a file from your project Git will track this for you too, and

change the symbol to a “**D**” - neat!! (Note because you are tracking these changes with Git, if you delete a file you can always “rewind the clock” and get the file back).



2. The two files that we didn’t add? These are hidden files and you can tell that by the . in front of them. The .DS_Store file is a MacOS file that we don’t want to track in Git or GitHub (it’s specific to your computer). That’s ok, we can just leave it in our Git tab and never stage it (it will remain there with a ? next to it). Alternatively, we can tell Git to ignore this file, which is what the .gitignore file is for! So let’s do that. At the same time we might tell Git to ignore some other files or folders that we may add in the future.

Ignoring files with .gitignore

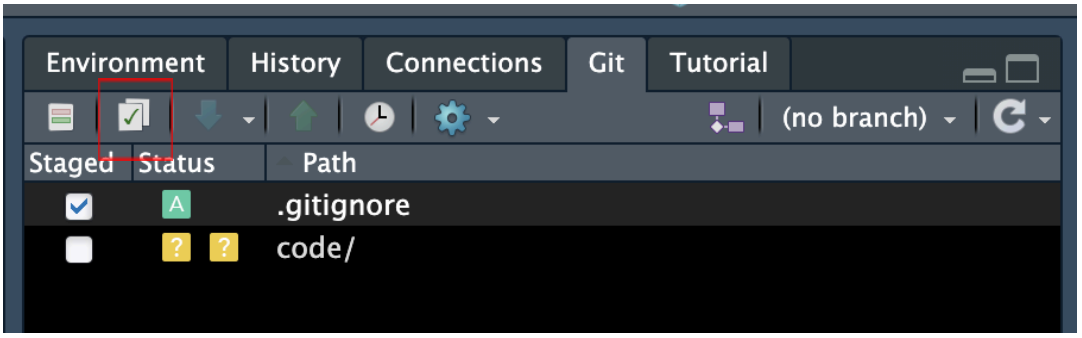
Go back to the class workshop project you cloned earlier and open the .gitignore file in there. You’ll see a few files listed. We’ll have a class discussion about what these are, why we might (or might not) want them to be ignored by Git, and then we can copy some of these to our new repo and save them in our own .gitignore file. Notice how once we do this, the .DS_Store file disappears from the Git tab (for Mac OS users only 🙂).

3.7 Working in Git and GitHub
Committing in Git

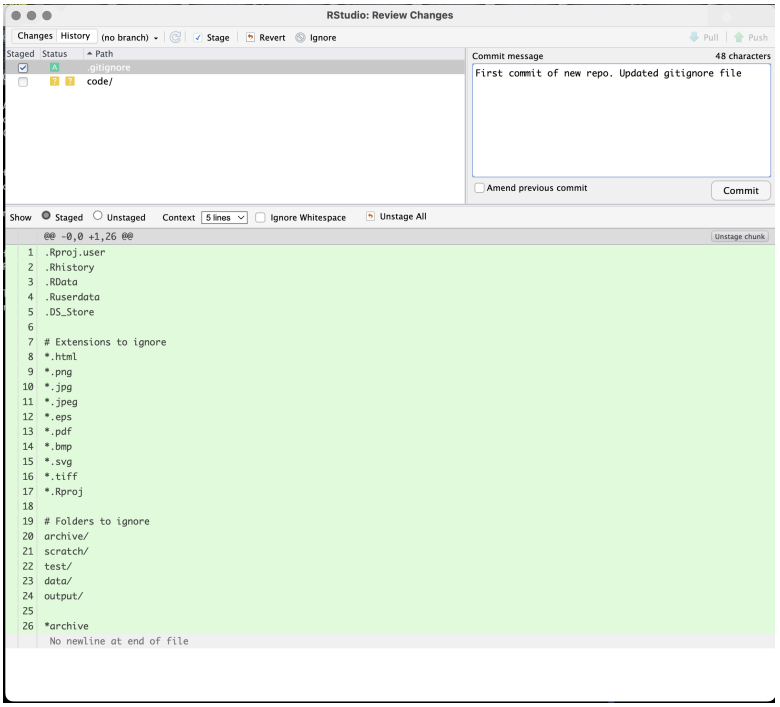
As with any computer work, once we’ve made some changes to a file, we need to save them. When you do this to a file that Git is already tracking, Git will notice, and notify you of this, by changing the symbol next to the file to an “M”, for Modified). So far all of our files are untracked, so they should still have a ? next to them in the Git tab. If we want Git to start tracking these files, we first need to *Stage* and then *Commit* them. Don’t worry, this may sound like a strange and complex process, but in reality it’s easy and we’ll work through this now together in the workshop.

The easiest way to *Stage* a file in RStudio is simply to check the box next to that file in the Git tab. Think of *Staging* as the process of deciding what files you are going to *Commit* in one go. Perhaps you have made a lot of saved changes to different files in your project, but you don’t want to commit them all at once. Instead, you can select the ones you want to commit together and just go through the commit process for those files only.

For example, here I have staged only the .gitignore file of my new repo, and have left everything else untracked (only for now!!):

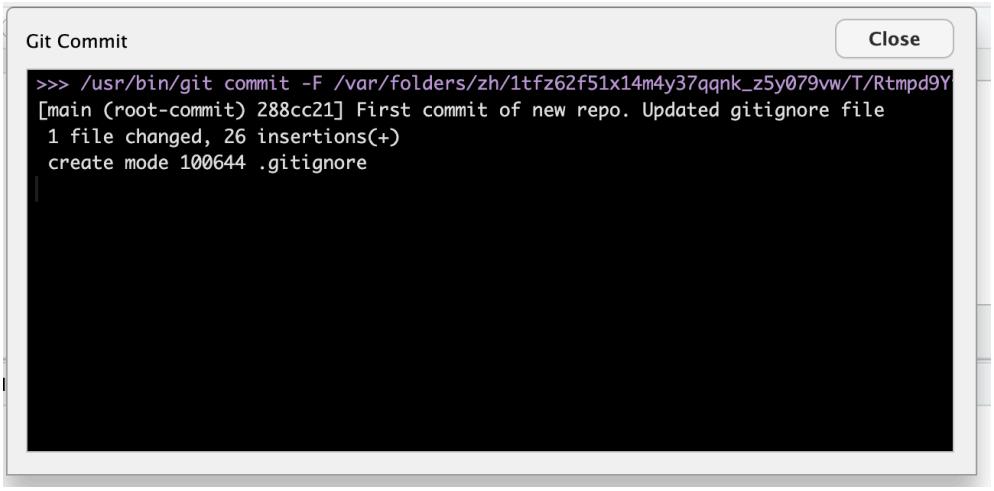


To commit this file and start tracking it in Git, all I now have to do is click on the tick button (in the red box above), which will make this pop-up screen appear:



Here I can (actually, must!) add a message to identify the commit and help collaborators (or just me, later on) know what this commit was all about. See what happens if you try to commit without adding a commit message…

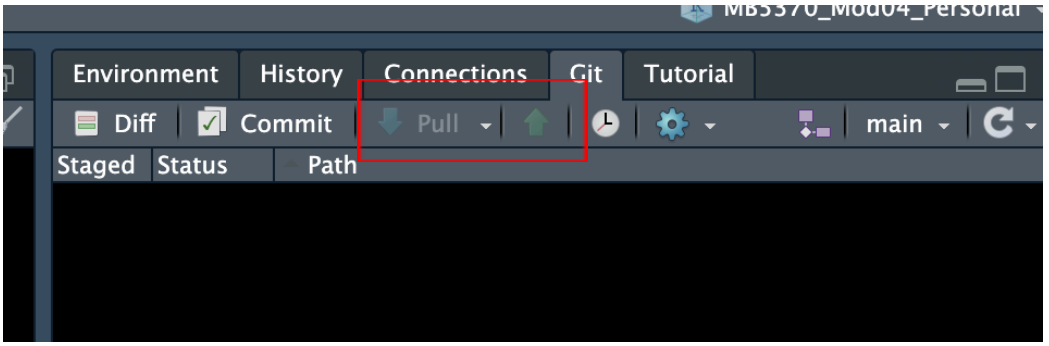
Once you've successfully committed you'll get another pop-up window to confirm this:



Notice this pop-up tells you the file path where the commit (i.e. version) has been stored, what has changed (in this case 1 file, with 26 lines of code added), and it repeats the message you used for the commit. It also has a serial number for the commit (288cc21, above). More on that later. For now, close the message pop-up and repeat the process with the remaining file(s). See what happens to the "Path" for the untracked "code/" file when you *Stage* it. Commit the remaining files with an appropriate message. Now we have all the files in our new repository committed and any future changes to these will be detected by Git and we can always roll back to this point if we ever need to.

Pushing to GitHub

So far, we have just been working locally on the one single version of this new project that now exists - on our own machine! You can tell this by looking at the buttons we would use to *Pull* from or *Push* to a GitHub remote:

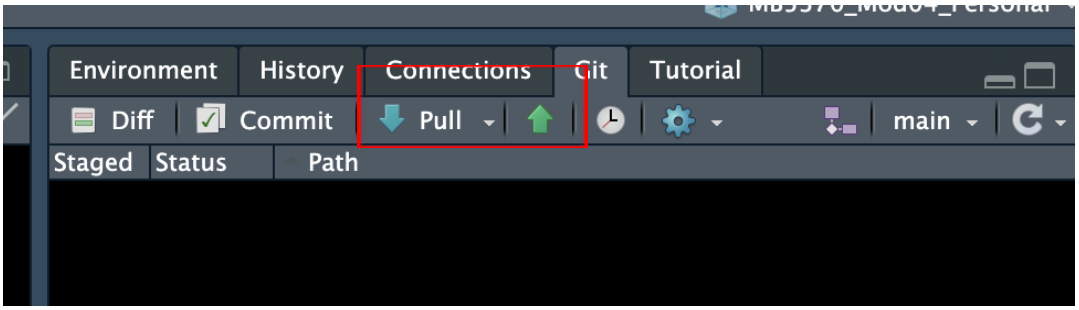


Note they are greyed out! But only for now. We are going to change that by *Pushing* this repository to GitHub and getting Git and GitHub to start talking to each other.

Because we already have GitHub accounts and this project is now being tracked by Git, this step is super-easy. In your console just run the following function:

```
usethis::use_github()
```

This one single line of code will replicate your entire project (well, the parts that Git is tracking) onto GitHub (aka the *Remote*). Neat! To confirm this worked you can look again at the Pull/Push buttons in the Git tab and see that these are now no longer greyed out!



What do you think will happen now, if you click on either of these? Have a go and find out! Report what happens to the rest of the class.

We can also tell that the usethis function worked because now we have a repository on GitHub! This should have popped up on your default browser when you ran this code but if not, head to your GitHub page to check it out.

3.8 Workflow with Git and GitHub

Congratulations! You now have a fully functioning project on your computer, being tracked by Git *and* linked to a remote GitHub repository, from where you can share your work with the world. Excellent! Now it's time to go to work, right!?!

Actually, at this point we just need to briefly cover how to manage your workflow in Git and GitHub. Of course, you are now going to start developing files and documents in this project. There will be figures generated and outputs like tables and lists that need to be stored. All of these need to be managed by your new version control friend, Git. But it needs your help.

Think back to how we discussed basic file management, without version control. We essentially just keep re-saving a file, perhaps updating the name once in a while, to help keep track. Something like this:

Edit → Save → Edit → Save → Edit → Rename/Save → Edit… and so on.

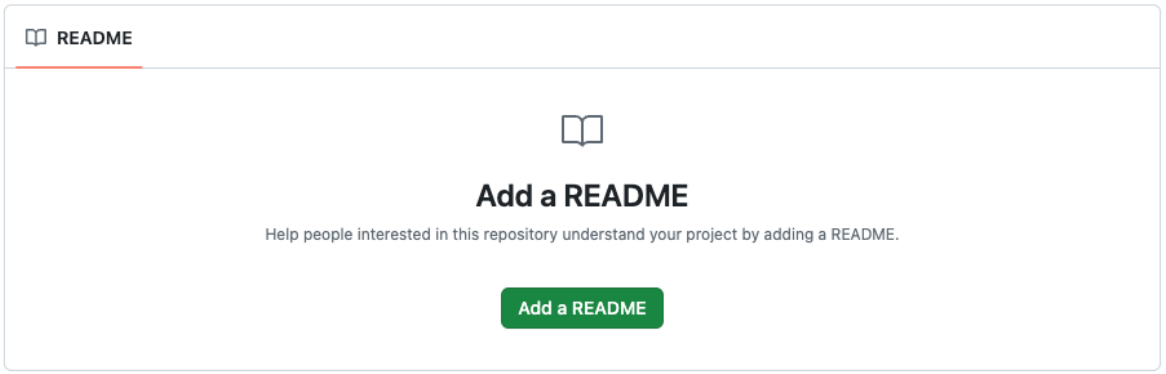
Now we have a powerful tool at our disposal and it changes this workflow a bit. No need to change the file name to indicate a new or different version (unless you have some other reason to). Now we need to incorporate Git and GitHub with the following steps:

Edit → Save → Stage → Commit → Pull → Push… and so on.

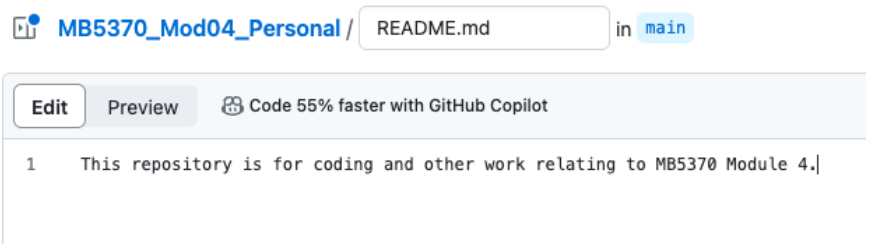
So far, we've conducted all these tasks. Except *Pulling*. We need to *Pull* when we are collaborating with others on a repo and there are changes we need to incorporate onto our own version from the remote.

To illustrate this, we'll just put it into practice! Of course, we don't yet have anyone else directly collaborating on this new project we just created (but we can cover how to do this if people are interested - just let me know!). BUT, we can make a change directly in our GitHub repo and then *Pull* it down to our local version.

First, navigate to the home page for your new GitHub repository. It's likely still open in your browser. Here you can see that GitHub is suggesting you add a README file. What a great idea! You *could* create a README file on your local version and Push it to the remote and that is likely the way that you will normally manage this sort of task. However, what we'll do now is click on "Add a README" in GitHub:



…and enter some explanatory sentences about what this repo is all about. I went with:



Next we need to save this file into the repo. You'll see in the top right corner we have the option to "Commit changes…" for this file (notice that within GitHub, we skip out the *Save* and *Stage* steps and go straight to *Committing*). Click the green button!

3.9 Progress review

We just covered a lot of material in getting set up with a version controlled project using Git and GitHub, but now we are ready to move on with the workshops and start exploring data visualisation and the other amazing things we can achieve with our data using these tools! Before we move on let's just review the key functions and concepts in using version control in RStudio, using Git and GitHub

Pull

Pulling a repository is like syncing your local project from the remote server (GitHub) onto your computer, and combining it with whatever you have on your computer. This is vital to do if someone else has done some work on it, or you're working on the same file from several computers. Make sure you Save, Stage and Commit your own work first though, or it may get overwritten!

Stage

Git *Stage* simply prepares your files to *Commit*. Best example for this I have seen is that it's like a dump truck, you fill it with the changes so that it's ready to move your work to the repository when you decide to *Commit* your changes. Here's a full list of these types of examples. You can decide which files you want to *Commit* by *Staging* just those files.

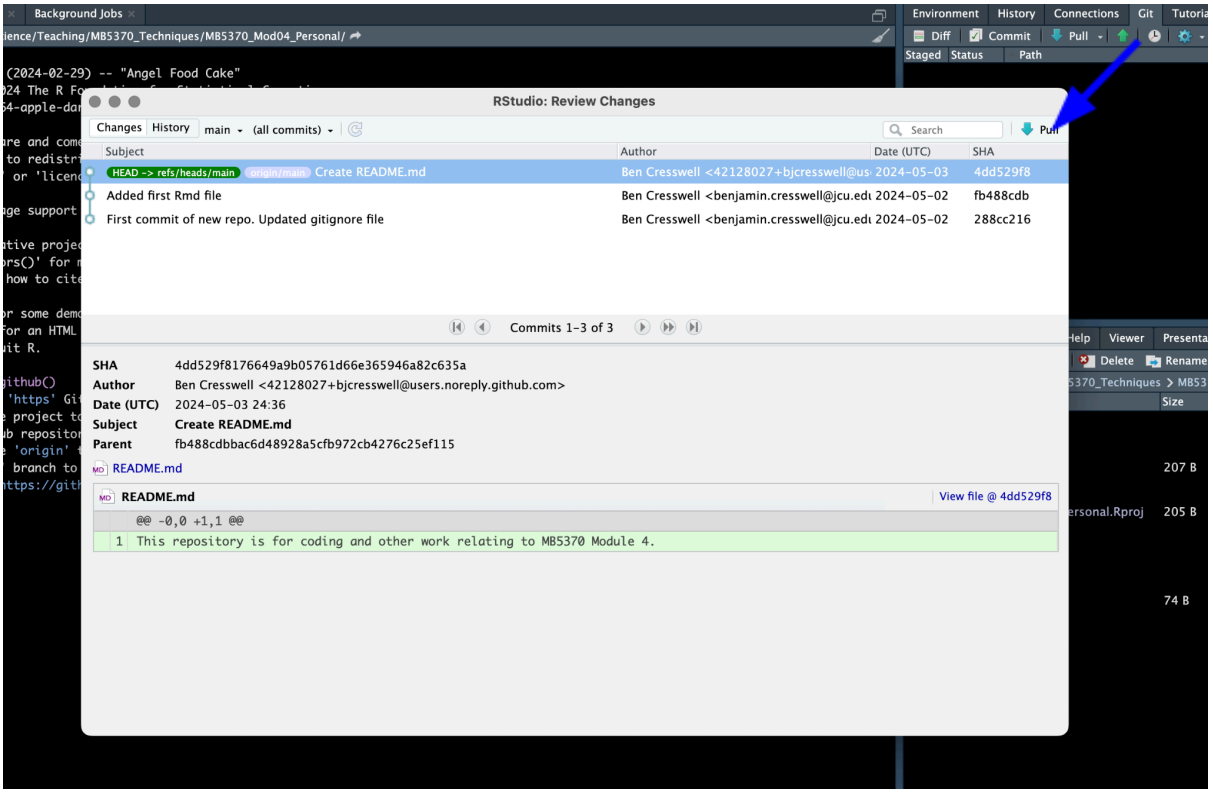
Commit

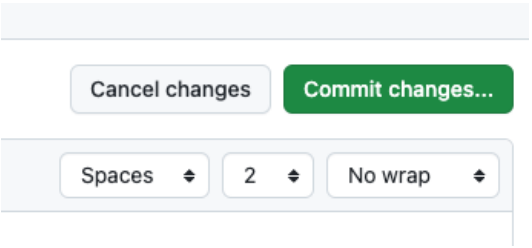
Git *Commit* is the function that saves this 'version' of your repo into your repository's version history. However, it's only saved locally at this point (not on your remote repository on the Github website). Make sure you include a meaningful human-readable Commit message of what you've done to your repo!

Push

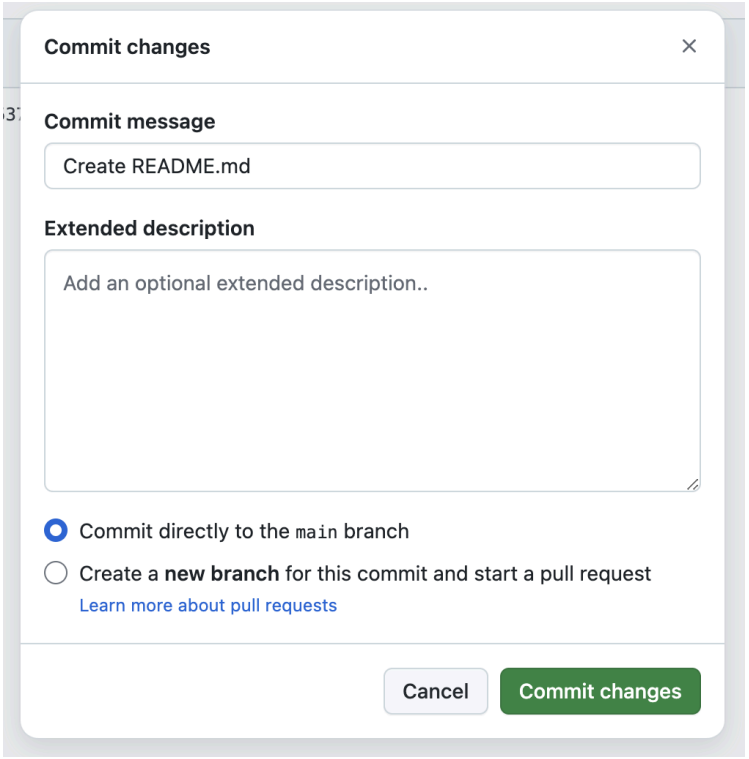
Once you are happy with your work and have *Committed* to Git locally, *Push* is the function that sends your changes to your remote server and synchronises with that version. If you want others to see the work you've done, or you want to save it properly as a backup, or access your work from a different computer, then you need to push your repo to Github. Remember you can *Commit* without *Pushing*, which can be useful if you want to track files locally before sending to GitHub.

If you want to see your Commit history, just click on the little clock to see all of your versions, and you can go back in time at any time you like.


Background Jobs
ience/Teaching/MB5370_Techniques/MB5370_Mod04_Personal/
(2024-02-29) -- "Angel Food Cake"
Environment
History
Connections
Git
Diff
Commit
Pull
Staged
Status
Path
RStudio: Review Changes
Changes
History
main
(all commits)
Search
Pull
Subject
Author
Date (UTC)
SHA
HEAD -> refs/heads/main
origin/main
Create README.md
Ben Cresswell <42128027+bjcresswell@us
2024-05-03
4dd529f8
Added first Rmd file
Ben Cresswell <benjamin.cresswell@jcu.edu
2024-05-02
fb488cdb
First commit of new repo. Updated gitignore file
Ben Cresswell <benjamin.cresswell@jcu.edu
2024-05-02
288cc216
Commits 1-3 of 3
SHA 4dd529f8176649a9b05761d66e365946a82c635a
Author Ben Cresswell <42128027+bjcresswell@users.noreply.github.com>
Date (UTC) 2024-05-03 24:36
Subject Create README.md
Parent fb488cdbbac6d48928a5cfb972cb4276c25ef115
README.md
README.md
View file @ 4dd529f8
@@ -0,0 +1,1 @@
1 This repository is for coding and other work relating to MB5370 Module 4.
Help
Viewer
Delete
Rename
Size
207 B
205 B
74 B



Just as when we are committing file changes within RStudio, we need to supply a Commit message. GitHub helpfully pre-fills this out for you:



You can change the commit message if you wish, but for now we'll leave it as-is and click on the green "Commit changes" button to… well, to commit the changes of course!

Now, we've committed a file change to our repository on GitHub. Excellent! So this change should be reflected in our local repository, right? After all, Git and GitHub are now talking to each other. Right? Well, actually, NO! We are still in charge here and it's up to us to decide when this happens.

Head back to your RStudio project and ponder those Pull and Push buttons again. What do you think will happen now if you click either of them? Have a go and let's discuss what happens.

Notice that when you Pulled, the README.md file now appears in the Files tab in the bottom right hand corner? Neat! You can even go into your OS file explorer app and you'll be able to see the new file there too.

Brilliant! So to recap, now you have:

- Set up Git on your computer and got a fully functioning GitHub account
- Cloned a repository from GitHub and started tracking it on your own machine
- Created a new project on your computer and started editing and tracking it using Git
- Got a backed up project and can rewind or undo back to any (committed) point in the project's history
- Pushed the project to a GitHub repo so that others can view your code and collaborate with you

Excellent work!

From here, it's just a case of repeating the steps in the workflow as often as you need to. "How often is that!?", I hear you ask. Well, like saving and backing up your work in general, Committing your work to Git should be conducted often! Git can't track a file if you don't Stage and Commit it, so the responsibility for doing this lies with you. Just stick to this workflow to avoid conflicts:

Edit → Save → Stage → Commit → Pull → Push

Question: what do you think will happen if you conduct these steps out of order? Let's discuss this as a class.

See here for more advice on workflow and best practice in Git and GitHub.

3.10 Assignments

Now you are up and running in GitHub and with Quarto in R Projects, you're ready to start tackling the assessment items for this subject.

The GitHub Professional Profile (20% of Total Grade)

Your task here is to ensure that you host a public repository for the contents of this module (you will have separate repos for future modules), cleanly mapping back to your local workspace progress scripts.

Action: Finalise your profile layout. Ensure your public-facing, personalized profile README.md page is updated and links to your repository tracking assets.

Execution Flow Reminder: Edit your code -> Save script -> Stage -> Commit -> Pull -> Push directly to your remote origin cloud storage!

The ePortfolio (40% of Total Grade)

You task here is to build your eportfolio over the course of this subject with one dedicated page per module, alongside a clean landing home page that features your professional career profile and image.

Action: Document your key technical takeaways, syntax strategies, and insights gained during each module, alongside embedded code.

The eportfolio is designed to allow you to showcase your work. Your website will include pages you develop during the modules you conduct for this subject, which you can customize and build out as much as you like (it could even ultimately form a full record of your experiences here at JCU.

At the end of the subject we'll cover how to get your own custom domain name (for around $10 per year, like our lab page for the Global Ecology Lab: www.globalecologylab.org), publish your website and get it indexed by Google if you want to!

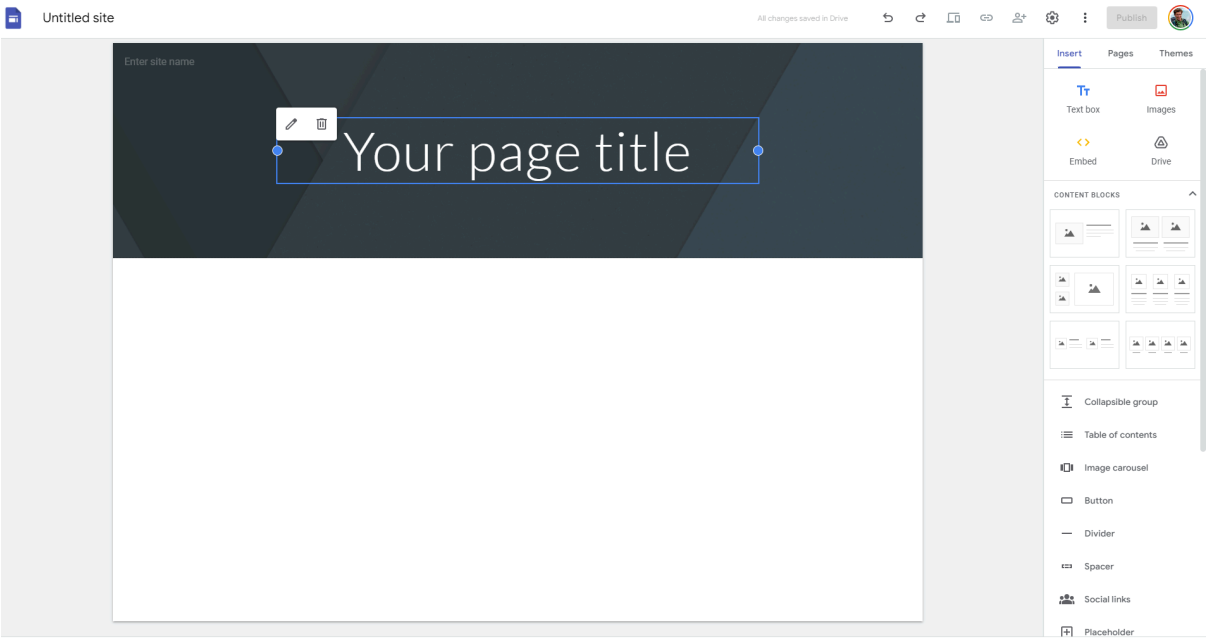
Your task, both for this and the remaining modules, is to advance this website, submit single pages at the conclusion of each module, and ultimately submit the final website as your eportfolio of work at the conclusion of the subject.

But how do you make a website?

First, you need to go to Google Sites. Anyone without a google account needs to make a new one before you'll be able to use Google Sites.https://sites.google.com/

Now click on the giant + symbol in the bottom right hand corner, which will create a new site.

Your new site will look like this.



Your task now is to customise it according to your own preferences.

As a minimum, your website needs to have:

1. A cover page, with the purpose of the site, a short paragraph about how this is a portfolio of your work for this subject,and perhaps some custom photos if you like. For an example of a site like this see here.
2. Play around with the themes tab at the right to try different things, you can change colours and formats.
3. Now, you will need at least one page for each module you conduct.

For some more inspiration, see how we've used Google sites in collaboration with others to deliver some of the datasets we've developed over the years:

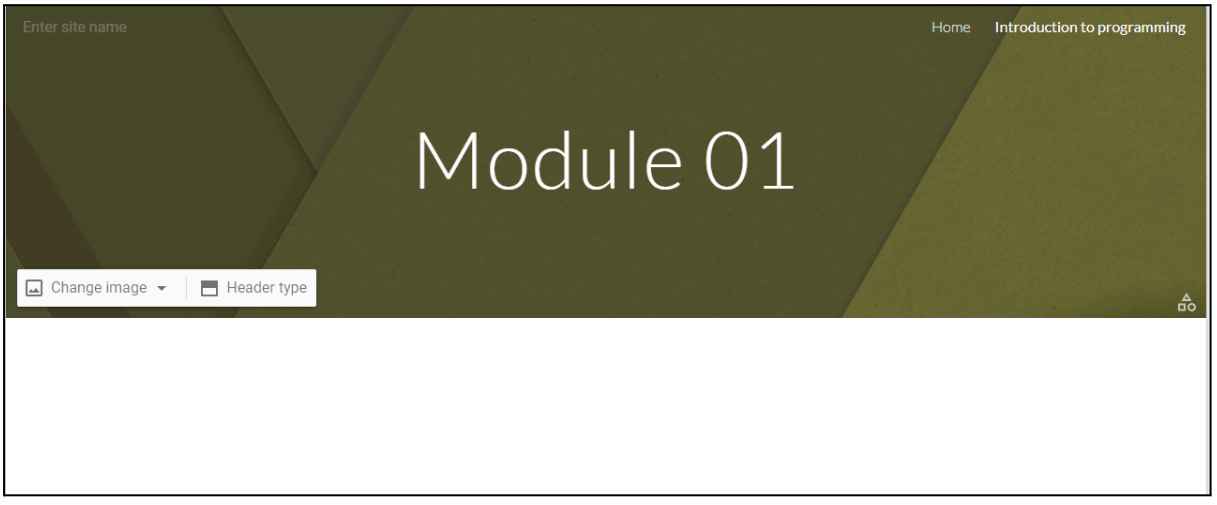
- www.globalintertidalchange.org
- www.intertidal.app
- www.globalecologylab.org

Once we've got our website set up we need to *embed* our code onto our site.

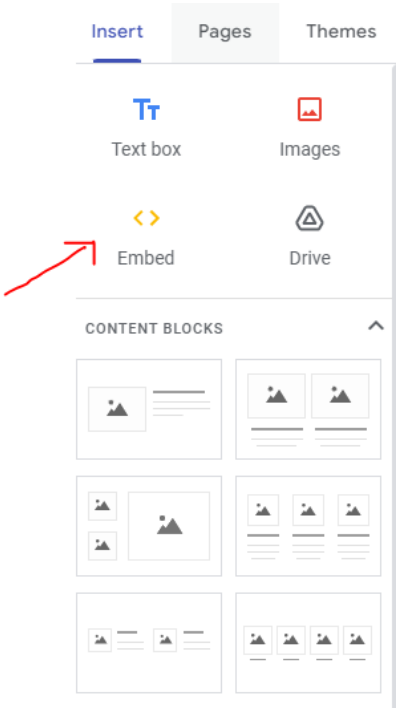
To do that, you need to render your qmd file (or knit your Rmd file) to an html file that you can embed in your Google Site. You can use a scripted workflow by following these instructions provided by rStudio (Rendering RMarkdown Files to HTML for Blogging). Note: in order to do this, we need to run the code in the console, or in a separate R script to the Rmd/qmd we are trying to render.

The other (easiest) way to do this is to click on the Knit or Render button at the top of the R Studio terminal. This will produce an html file in your workspace. Open it (use the Editor option to view it inside RStudio) and see what it looks like. This is the html code that you can paste into your Google Site.

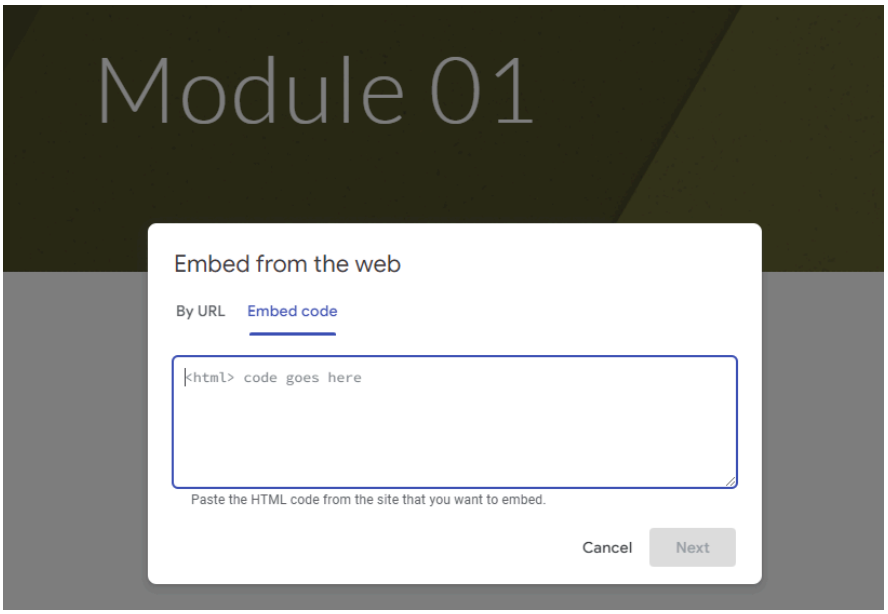
Now the fun bit. Go to your Google Site. Make a new page (use the Pages tab) to make your module page. Note here we've called the page introduction to programming, and given it a header Module 01. You can decide how you want to present yours.



You will then need to paste your html code into your page. You can do this by *embedding* the html code into your site. Click the `< > Embed` button.



You are now able to paste your html code into here.



Finally, in order for us to see the contents of your eportfolio you need to Publish it and make it viewable by anyone with the link. Use the two buttons at the top to do this:.

Change the access to “Anyone with the link”.

Workshop 4 – The Developer's Toolbox & AI

4.1 Session schedule

Data science and marine ecology workflows move rapidly. In this final workshop, we will explore tools that automate lengthy workflows, handle complex data-driven decisions, and leverage cutting-edge tools to accelerate your programming.

We will introduce GitHub Copilot and the chattr package to set up a modern AI-assisted workspace. From there, we will explore control structures—discovering how base R handles decisions and repetitions via if-else loops alongside modern tidyverse functional approaches via the purrr package. Finally, we will wrap these concepts into custom functions and close with an essential milestone review of your ePortfolio and GitHub profile milestones.

Time	Session Title	Focus

Hour 1	**Building Your Coding Co-Pilots**	Configuring educational access, initialising chattr and GitHub Copilot in RStudio, mapping UI shortcuts, and prompt engineering.
Hour 2	**Conditional Logic: Making Code Decide**	Building if and if-else structures vs. vectorised dplyr::if_else() and dplyr::case_when() pipelines.
Hour 3	**Automation: Iterating Efficiently**	Traditional for-loops running parallel to functional programming mapping alternatives with purrr.
Hour 4	**Custom Functions & Course Milestones**	Packaging reusable logic and completing critical ePortfolio / GitHub profile requirements.

4.2 Integrating AI into Your Coding Workflow

Rather than replacing the programmer, AI tools can act as "co-pilots," allowing you to focus on the biological and statistical logic while the AI handles boilerplate syntax.

We will configure two distinct AI tools directly within RStudio to establish a comprehensive, dual-layered coding workflow:

1. **GitHub Copilot:** An inline auto-completer that predicts the next chunk of code as you type.
2. **chattr:** A conversational reasoning engine embedded directly in your RStudio environment, capable of reading your loaded data sets and troubleshooting errors.

4.3 GitHub Copilot (Inline Autocomplete)

GitHub Copilot functions as a text-prediction engine. It is exceptionally useful for rapidly generating repetitive tidyverse pipelines or standard ggplot2 formatting.

Acquiring Educational Access

While GitHub Copilot is a paid service, students receive free access through the GitHub Education program.

1. Navigate to the GitHub Education portal using your web browser.
2. Apply for the Student Developer Pack using your university email address.
3. Once approved, GitHub Copilot will be permanently linked to your GitHub account.

Activating Copilot in RStudio

1. Open RStudio and navigate to Tools > Global Options > Copilot.
2. Check the box to Enable GitHub Copilot.
3. Click Sign In. Copy the verification code provided and paste it into the GitHub authorization window that opens in your browser.

Exercise: AI-Driven Data Visualisation Challenge

Now that GitHub Copilot is active inside your RStudio session, we can use it to revisit the data visualisation skills you learned in Workshop 2. Instead of writing the code manually from memory, your goal here is to practice using inline text comments to guide Copilot into building the code chunks for you.

We will use the Palmer Penguins dataset (palmerpenguins), which records biological and environmental metrics for three penguin species in the Palmer Archipelago. This is a real life data set from a real research station in the Antarctic, run by the US National Science Foundation. You can read more about it here.

Step 1: Prepare Your Workspace

If you are already working in an Rmd or qmd file, keep working there. If not, make a new file and save it into your project directory.

Create a fresh R code chunk. Inside that chunk, write a hash comment prompting Copilot to load the core packages. Type the comment exactly as shown below, hit Enter, and press Tab to accept Copilot's code prediction:

```
# Load tidyverse and palmerpenguins libraries
```

It should have auto-filled the following code:

```
library(tidyverse)
library(palmerpenguins)
```

You can prompt Copilot for a multitude of programming options in this way

Step 2: Prompt for a Multi-Variable Scatter Plot

Skip a line within your code chunk, then write a clear, sequential prompt instructing Copilot to generate a scatter plot tracking physical traits across different species.

```
# Create a ggplot scatter plot using the penguins dataset.
# Plot bill_length_mm on the x-axis and bill_depth_mm on the y-axis.
# Color points by species and use geom_point with size 3.
# Add a minimal theme and clean labels.
```

Action: Press Enter at the end of the final comment line and wait for Copilot to display the predicted code in gray text. Press Tab to accept it, then run the code chunk to view your plot.

Step 3: Open-Ended Comparative Plotting

To truly test how inline autocomplete adapts to your instructions, choose one of the options below. Create a new code chunk in your document, write your own multi-line comment script to describe the exact plot you want, and see if Copilot can build the syntax. Remember Copilot is just a tool - you are still the programmer, so don't accept suggestions blindly!!

- **Option A (Distribution Check):** Instruct Copilot to generate a boxplot comparing flipper_length_mm across different species, faceted by island, using custom color palettes.
- **Option B (Biomass Correlation):** Instruct Copilot to build a scatter plot mapping body_mass_g against flipper_length_mm, adding a linear regression trend line (geom_smooth) for each species.

```
#[Your Turn: Write 3-4 descriptive lines here explaining your choice]

# Next, hit Enter, let Copilot predict the ggplot pipeline, and press
Tab to accept.
```

Step 4: Verify and Document

Did it work!? If not, go back and try to either fix your prompt, or better yet, look at the code and see if you can easily spot some adjustments you can make, based on your experience gained so far. This is the best way to use AI or a Copilot tool - both you and the tool working together! Once you are happy with your plot(s), make some notes about your decisions and how you used Copilot, then save your Rmd or qmd file. (Don't forget to Commit and Push to your GitHub repo too!!)

Take a screenshot of your favorite generated graphic. You will use this image on your ePortfolio page for this module to show how you used automated co-pilots to accelerate

reproducible data workflows.

4.4 *chattr* (Conversational Workspace Engine)

Unlike generic web-browser chat interfaces, the chattr package is fully integrated within RStudio, meaning it can securely read metadata from your active environment to provide contextual support.

Installation

Like any package chattr needs to first be installed before you can load and use it. Install chattr along with its core connectivity dependencies by running the following commands in your console:

```
install.packages("ellmer")
install.packages("chattr")
```

Generating an API Key

To power the conversational engine, you must connect it to a Large Language Model (LLM). You can choose between Google Gemini, OpenAI's ChatGPT, or Anthropic's Claude. Each requires an Application Programming Interface (API) key.

LLM Provider	Where to Get the API Key	Notes for Setup
Google Gemini	Google AI Studio	Click "Create API Key". You may need to manually select to create a new "Google Cloud Project" from the dropdown menu first. Gemini Flash models are extremely fast for coding.
OpenAI (ChatGPT)	OpenAI Developer Platform	Requires setting up a billing account with a small minimum balance (e.g., $5) to activate API usage, even for basic models like GPT-4o-mini.
Anthropic (Claude)	Anthropic Console	Also operates on a pay-as-you-go API structure. Claude 3.5 Sonnet is highly capable for data science tasks.

Securely store your API key

API keys act as passwords and must never be saved directly in your R scripts. We store them safely in the R environment file.

1. Run the following command in your R console: `usethis::edit_r_environ()`
2. A new file named .Renviron will open. Add **one** of the following lines, depending on the model you chose, pasting your actual key between the quotation marks:

```
# Option A: For Google Gemini
GEMINI_API_KEY="your-long-api-key-here"

# Option B: For OpenAI ChatGPT
OPENAI_API_KEY="your-long-api-key-here"

# Option C: For Anthropic Claude
ANTHROPIC_API_KEY="your-long-api-key-here"
```

3. Save the file and **restart your R session** (Ctrl+Shift+F10 on Windows/Linux or Cmd+Shift+0 on Mac).

Connecting chattr to Your Model

To ensure chattr always utilizes your preferred model without needing manual re-connections every session, we will configure it inside your permanent R profile.

Run the following command to edit your user profile:

```
usethis::edit_r_profile()
```

Add one line corresponding to your chosen LLM provider to the file that opens up:

```
# Option A: For Google Gemini
options(.chattr_chat = ellmer::chat_google_gemini())

# Option B: For OpenAI ChatGPT
options(.chattr_chat = ellmer::chat_openai())

# Option C: For Anthropic Claude
options(.chattr_chat = ellmer::chat_claude())
```

Save the file and restart your R session (Ctrl+Shift+F10 on Windows or Cmd+Shift+0 on Mac).

Launching and Mapping the User Interface

To interact with the AI, you can run chattr::chattr_app(). For an optimal workflow, map it to a keyboard shortcut.

4. Navigate to Tools > Modify Keyboard Shortcuts.
5. Search for Open Chat.
6. Click inside the "Shortcut" column and press your desired combination (e.g., Alt+Shift+C).
7. Click Apply.

Exercise: Prompting for Context

Open your newly configured chattr interface using your keyboard shortcut. Paste a dataset summary or ask a conversational question to test the model delivery workflow. For example, prompt the interface with: *"Show me how to load the built-in iris dataset and use ggplot2 to build a scatter plot of Sepal.Length versus Sepal.Width colored by Species."* Use the **Copy to Document** button to drop the code directly into an active R script and execute it.

4.3 Conditional Logic - filtering your data

Computers excel at evaluating parameters and taking specific routes based on the answer. In data science, this is how we filter outliers, classify categorical variables, or tag environments by environmental stress limits.

Base R: if and else

The classic base R conditional structure screens a single value at a time. Its basic anatomy uses round brackets () for the logical condition and curly braces {} for the action if that condition is true.

```
# Simple Example: Checking Sea Surface Temperature (SST)
sst <- 30.2

if (sst > 29.5) {
  print("Warning: Marine heatwave threshold exceeded!")
} else {
  print("SST remains within baseline parameters.")
}
```

Tidyverse Alternative: Vectorised Conditionals (dplyr)

Base R if and if-else blocks break down when you pass them an entire column (a vector) of data. They only look at the first item. Because we are already familiar with the tidyverse, we can use vectorised functions that evaluate conditions across complete rows instantly. These functions come from the powerful data-wrangling package dplyr, that comes as part of the tidyverse.

if_else()

The if_else(condition, true_output, false_output) function evaluates an entire column item-by-item:

```
# Load tidyverse (if not already loaded)
library(tidyverse)

# Sample coral data
coral_monitoring <- tibble(
  site = c("Site_A", "Site_B", "Site_C"),
  depth_m = c(12, 35, 8)
)

# Classify depth using if_else
coral_monitoring <- coral_monitoring %>%
  mutate(zone = if_else(depth_m > 30, "Deep Reef", "Shallow Reef"))
```

Task: what do you think that function `mutate()` does? How can we get more information about it? What about the `%>%` symbol? Use your developing programmer skills to find out more information about these two functions and record your findings now in your qmd.

case_when()

When you have more than two possibilities, stacking multiple if-else blocks becomes unreadable. case_when() provides a clean, sequential evaluation layout using formula tildes (~):

```
coral_monitoring <- coral_monitoring %>%
  mutate(reef_category = case_when(
    depth_m < 10  ~ "Lagoon / Flats",
    depth_m <= 30 ~ "Crest / Slope",
    depth_m > 30  ~ "Mesophotic / Deep",
    TRUE          ~ "Unclassified" # Catch-all remainder
  ))
```

Task: Environmental Stress Classification

Using the chattr app or inline copilot, your task here is to write an R script that initialises a tibble named marine_stations containing a column for salinity with values 35, 28, 32, and 12. Use the pipe operator (%>%) and mutate() combined with case_when() to create a new column named environment_type. Classify values below 15 as "Estuarine", values between 15 and 30 as "Brackish", and values above 30 as "Marine". Run your script and verify the table structure.

4.4 Automation — Iterating Efficiently

Automation means instructing R to repeat a task across multiple items without manual copy-pasting. We will cover the foundation (for-loops) and the elegant tidyverse workflow (purrr) side-by-side.

Base R: for-loops

A for-loop repeats a code chunk for each element in a designated sequence.

```
# Classic loop anatomy
for (year in 2020:2024) {
  print(paste("Processing climate data for year:", year))
}
```

If we want to track real-world data, loops can iterate over file directories (e.g., reading 12 monthly turtle tracking CSVs sequentially):

```
# Loop across index sequences
transect_lengths <- c(50, 100, 25, 75)

for (i in seq_along(transect_lengths)) {
  print(paste("Transect number", i, "measures", transect_lengths[i],
"metres."))
}
```

Tidyverse Alternative: Functional Programming (purrr)

While for-loops are a core programming construct, they require you to manage index variables (i) and explicitly set up empty "containers" to store results, which often leads to verbose code and hidden bugs.

The tidyverse handles iteration via the purrr package using map functions. These map a specific operation onto every item in a list or vector, automatically ensuring that your output matches the precise data structure you expect.

Function	Expected Output Data Type
map()	Returns a flexible List structure
map_dbl()	Returns a vector of Decimals / Numeric values
map_chr()	Returns a vector of Text strings / Characters
map_df()	Returns a combined Data Frame / Tibble

Comparison Example: Calculating Square Roots of Site Vectors

Look at how much cleaner functional mapping behaves compared to building a manual container loop:

Using a standard for-loop:

```
site_areas <- c(144, 400, 625)
results <- numeric(length(site_areas)) # Must build an empty container first

for(i in seq_along(site_areas)) {
  results[i] <- sqrt(site_areas[i])
```

Using the purrr equivalent:

```
library(purrr)
# Map directly and define your expected output type explicitly
mapped_results <- map_dbl(site_areas, sqrt)
```

There is no right or wrong approach - either base R or tidyverse methods provide the same result. A good idea is to figure out which method works best for you and then thoroughly familiarise yourself with the syntax and structure of the code.

Iterating Across Complex Groups

You can combine purrr pipelines seamlessly inside data frames to split analysis across separate groups:

```
# Iterate a summary function over split lists of data
iris %>%
  split(.$Species) %>%
  map(~summary(.x))
```

Task using chattr:

Use chattr to convert a complex operation. Prompt the engine: "I have a list of three vectors containing fish count data. Write a for-loop to calculate the mean of each, and then show me the exact parallel way to do it using purrr's map_dbl function." Compare the visual simplicity.

4.5 Writing custom functions

When you find yourself copying and pasting the exact same block of logical code or loop transformations three or more times, it is time to write a custom function. Functions package your logic into clear, named, reusable assets.

Function Construction Blueprint

A function consists of a Name, Arguments (the input variables), and a Body (the execution logic wrapped inside curly braces).

```
# Function Definition
calculate_coral_mortality <- function(initial_count, surviving_count) {

  # Logic safety switch using our conditional tools!
  if (initial_count <= 0) {
    stop("Initial count must be greater than zero.")
  }

  mortality_rate <- (initial_count - surviving_count) / initial_count
  return(mortality_rate)
}

# Utilizing your custom function
calculate_coral_mortality(initial_count = 120, surviving_count = 84)
```

Best Practice Rule: Always test your functions in a clean local environment. Ensure they rely strictly on the arguments passed directly into them, rather than accidentally grabbing random background objects floating in your global R environment.

Student Exercise 4: Custom Function Construction Task

Build a custom function named convert_temp_c_to_f that accepts a single argument representing temperature in Celsius. The function should perform the math conversion: F = (C * 9/5) + 32. Include a conditional safety check using an if statement that halts execution via stop() if the entered value is below absolute zero (-273.15 degrees Celsius). Test your function with both a valid temperature and an invalid value to ensure the error handling behaves as expected.

4.6 Advanced Scripting Extension Challenges

For anyone seeking an extra challenge, apply your structural programming tools to execute these two advanced production pipelines.

Challenge 1: Nested Multi-Vector Iteration

In real-world field surveys, calculations often depend on matching values across multiple lists or matrices. Read the documentation for purrr::map2_dbl(). Given a vector of raw survey counts counts <- c(15, 24, 8, 42) and a vector of species-specific scaling factors factors <- c(1.2, 0.8, 2.5, 1.1), write a mapping statement that iterates across both vectors simultaneously to return a single vector of scaled abundance values. Do not use a manual index loop.

Challenge 2: The Robust Data Wrangling Pipeline Function

Create a custom function named clean_and_classify_survey that handles complex data frame structures. The function must fulfill these criteria:

- Accept a data frame and a column name threshold parameter as inputs.
- Safely check if the specified column name exists within the passed data frame; if it does not, trigger a user-friendly error message using stop().
- Filter out rows containing missing values (NA) within that specific column.
- Use the pipe operator and mutate() along with a vectorised conditional (if_else() or case_when()) to append a status string variable classification based on your threshold parameter.
- Return the cleaned, altered data frame to the global environment.

Test your custom pipeline using a mock tibble that contains at least one missing value to confirm your code handles anomalies without crashing.

4.7 Module wrap up!

Congratulations! That concludes the workshops for our Programming Fundamentals workshops. By completing these sessions, you have laid the foundations for your future careers as highly-skilled marine scientists and progressed from absolute programming fundamentals to managing an AI-assisted, professional data science workflow.

Take a moment to review the comprehensive technical foundation you have built across all four workshops:

Workshop 1 – Foundations of R and Data Structures

You began by establishing your core programming workspace. You learned to navigate the RStudio environment, manipulate variables, and interact with fundamental data structures like vectors, matrices, lists, and data frames. This session built the muscle memory needed to access, index, and organize basic biological data.

Workshop 2 – Data Visualisation with ggplot2

You transitioned into visual storytelling and exploration using the tidyverse ecosystem. By utilising the grammar of graphics, you can now bind data columns to aesthetic mappings, manipulate geometric layers, and handle complex multi-panel charts using facets. This toolset establishes the foundations for you to visually audit data distributions and uncover ecological patterns. We'll be building on this in our R for Marine science module.

Workshop 3 – Version Control & Portfolio Deployment

Stepping away from syntax, we covered how professional software projects are organised, tracked, and stored. You initialised local project directories using Git and established a professional backup and collaboration loop with remote GitHub repositories.

Workshop 4 – The Developer's Toolbox & AI

In this final workshop, we brought our data science toolkit into the modern era. Now you are set up with embedded autocomplete engines like GitHub Copilot and conversational contexts like chattr directly inside your IDE, using natural language to accelerate your programming. You then combined base R control structures with vectorized tidyverse alternatives like dplyr and functional iteration via purrr to build scalable, automated, and error-safe analytical custom functions.

Take the final 15 minutes of today's session to secure your hard work and format it directly into your assessment platforms.